УЧЕБНИК · GNU/LINUX DEBIAN · BASH

Bash для арабо-русского словаря в базе данных

Подробное руководство для пользователя Debian GNU/Linux: от базовых команд до полноценного проекта словаря с SQLite, PostgreSQL и MariaDB

Для кого эта книга

Учебник рассчитан на читателя, который уже умеет выполнять простейшие команды (`cd`, `ls`, `mkdir`) в терминале Debian GNU/Linux, но не писал собственных скриптов и не работал с базами данных из командной строки. Вы пройдёте путь от первого `#!/bin/bash` до полноценного Bash-проекта арабо-русского словаря, хранящегося в реляционной БД.

Что внутри

Двадцать глав и приложение построены по принципу «от простого к сложному». Сначала — основы Bash: переменные, перенаправления, циклы, функции, регулярные выражения. Затем — настройка Debian для работы с арабским письмом, сравнение трёх популярных СУБД, проектирование схемы словаря и сборка готового проекта с бэкапами и импортом из CSV. Все примеры проверены на Debian 12 (bookworm).

Как читать

Главы можно читать подряд или как справочник. Каждая глава заканчивается практическим заданием, а ключевые идеи выделены в рамки. Код в тёмных блоках — это сеансы терминала, в светлых — содержимое скриптов. Кавычки и тире в тексте типографские, в коде — ASCII.

ПЛАТФОРМА	Debian GNU/Linux 12 (bookworm)
ОБОЛОЧКА	Bash 5.2+
СУБД	SQLite 3 · PostgreSQL 15 · MariaDB 10.11
УРОВЕНЬ	Базовый → Средний
ИЗДАНИЕ	2026 · Z.ai

Оглавление

СОДЕРЖАНИЕ · 20 ГЛАВ + ПРИЛОЖЕНИЕ

ЧАСТЬ I. ОСНОВЫ BASH

01 Знакомство с Debian и командной строкой 3

02 Базовые команды для работы с файлами 5

03 Переменные, строки и параметры 8

04 Ввод-вывод и перенаправления 11

05 Условия и ветвление 14

06 Циклы 17

07 Функции 20

08 Текстовые утилиты: grep, sed, awk 23

09 Регулярные выражения 26

ЧАСТЬ II. АРАБСКИЙ ЯЗЫК И DEBIAN

10 Арабские шрифты и ввод в Debian 29

ЧАСТЬ III. БАЗЫ ДАННЫХ

11 Сравнение СУБД для словаря 32

12 SQLite: установка и основы 34

13 PostgreSQL: установка и основы 36

14 MariaDB: установка и основы 38

15 SQL-схема словаря 40

ЧАСТЬ IV. ПРОЕКТ СЛОВАРЯ

16 Bash-скрипты для работы со словарём 42

17 Импорт данных из CSV 45

18 Обработка ошибок и безопасность 47

19 Полный проект словаря и бэкап 50

20 Итоги и дальнейшее изучение 53

A Приложение. Шпаргалка по командам 55

ГЛАВА 01 · ОСНОВЫ

Знакомство с Debian и командной строкой

Debian GNU/Linux — один из старейших и наиболее уважаемых дистрибутивов Linux. Основанный в 1993 году Иэном Мёрдоком, Debian славится стабильностью, тщательным тестированием пакетов и полностью свободным программным обеспечением. На Debian построены десятки других дистрибутивов: Ubuntu, Linux Mint, Kali Linux, Raspbian для Raspberry Pi. Если вы работаете в Debian, ваши навыки переносятся на всю экосистему Debian-подобных систем.

Для разработки Bash-скриптов и работы с базами данных Debian подходит идеально. В стандартной поставке уже есть Bash, основные утилиты GNU (coreutils), текстовые редакторы (nano, vim), система пакетов apt. Установка дополнительных компонентов — SQLite, PostgreSQL, MariaDB, арабских шрифтов — выполняется одной командой и не требует сложной настройки.

Командная строка vs графический интерфейс

Графический интерфейс (GUI) удобен для повседневных задач: браузер, текстовый редактор, просмотр фотографий. Но для автоматизации, работы с серверами и разработкой скриптов командная строка (CLI) несравненно мощнее. Причин несколько. Во-первых, команды можно записывать в скрипт и повторять тысячи раз без ручного вмешательства. Во-вторых, команды легко комбинировать через конвейеры, получая сложную обработку данных из простых кирпичиков. В-третьих, командная строка работает по SSH на удалённых серверах без графической оболочки — а большинство серверов именно такие.

В Debian 12 (bookworm) по умолчанию устанавливается рабочий стол GNOME, но терминал всегда доступен через комбинацию `Ctrl+Alt+T` или поиск по приложению «Терминал». Если вы установили Debian без GUI (серверный вариант), вы окажетесь в чистом текстовом режиме — это нормально, всё, что описано в книге, работает и там.

Запуск терминала и первая команда

Откройте терминал. Вы увидите приглашение вида `user@debian:~$`. Это и есть командная строка Bash. Здесь `user` — имя пользователя, `debian` — имя хоста, `~` — текущая директория (домашняя), `$` — приглашение обычного пользователя. Если стоите под root, будет `#` вместо `$`.

```
user@debian:~$ echo "Привет, Debian!"
Привет, Debian!
user@debian:~$ bash --version | head -1
GNU bash, версия 5.2.15(1)-release (x86_64-pc-linux-gnu)
```

Команда `echo` печатает свой аргумент на экран. Команда `bash --version` показывает версию Bash, а вертикальная черта `|` передаёт её вывод в `head -1`, которая оставляет только первую строку. Так выглядит конвейер — основная идиома Unix.

Права суперпользователя: su и sudo

Многие системные операции — установка пакетов, редактирование конфигурации в `/etc`, запуск служб — требуют прав root. В Debian есть два способа получить эти права. Первый — команда `su` (switch user): после ввода пароля root вы переключаетесь в его контекст. Второй — `sudo`, которая выполняет одну команду с правами root. По умолчанию в Debian `sudo` может быть не настроен для вашего пользователя.

```
user@debian:~$ su -
Пароль:
root@debian:~# apt update && apt upgrade -y
...
root@debian:~# exit
user@debian:~$
```

Чтобы включить `sudo` для своего пользователя, выполните от root: `adduser user sudo`, затем перезайдите в систему. После этого можно писать `sudo apt install sqlite3` вместо `su -` + команда + `exit`. В этой книге мы будем использовать `sudo` — это общепринятый стиль.

Обновление системы

Перед началом работы обновим список пакетов и саму систему. Это рекомендуется делать после установки и регулярно thereafter. Команда `apt update` обновляет индексы пакетов из репозиториев, `apt upgrade` — устанавливает новые версии уже установленных пакетов.

ОБНОВЛЕНИЕ СИСТЕМЫ BASH

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y bash-completion vim git curl
```

Пакет `bash-completion` добавляет автодополнение по клавише `Tab` для команд и опций — бесценная экономия времени. `vim` — мощный текстовый редактор (альтернатива `nano` для новичков). `git` и `curl` — инструменты разработчика, пригодятся позже.

> **СОВЕТ**
>
> Если команда `sudo apt` ругается на «Permission denied», значит ваш пользователь не в группе sudo. Выполните `su -`, введите пароль root, затем `adduser ваш_логин sudo` и перезайдите в систему.

Структура книги и约定

В этой книге команды, которые вы вводите в терминал, показаны в тёмных блоках с зелёным приглашением `user@debian:~$`. Содержимое скриптов — в светлых моноширинных блоках с подсветкой синтаксиса. Имена файлов и команд в тексте выделены как `так`. Важные замечания — в рамках с заглавием «Совет», «Предупреждение» или «Практика».

Все примеры рассчитаны на Debian 12 (bookworm), но будут работать и в Debian 11 (bullseye), и в Ubuntu 22.04+ / 24.04+, и в большинстве производных дистрибутивов. Если вы на другой системе (Arch, Fedora), имена пакетов и пути могут отличаться — уточняйте в документации вашего дистрибутива.

ГЛАВА 02 · ФАЙЛЫ

Базовые команды для работы с файлами

Файловая система Linux — это дерево с корнем в `/`. Все диски, флешки, сетевые ресурсы монтируются как подкаталоги этого дерева. В отличие от Windows, нет букв дисков (C:, D:) — всё единая иерархия. Понимание этой структуры необходимо для эффективной работы в Bash.

Ключевые каталоги

ПУТЬ	НАЗНАЧЕНИЕ
`/`	Корень файловой системы
`/home/user`	Домашний каталог пользователя (сокращение `~`)
`/etc`	Конфигурационные файлы системы
`/usr`	Программы пользователя, документация
`/var`	Переменные данные: логи, базы данных, кэш
`/tmp`	Временные файлы, очищаются при перезагрузке
`/bin`, `/sbin`	Системные утилиты (в Debian 12 это символические ссылки на `/usr/bin`, `/usr/sbin`)

Для нашего проекта словаря важны три каталога. `~/dict-ar-ru` — рабочая папка проекта, где будут скрипты и БД. `/etc` — сюда иногда кладут общесистемные конфиги (но для личного словаря это не нужно). `/usr/share/fonts` — системные шрифты, включая арабские.

Навигация: pwd, cd, ls

Команда `pwd` (print working directory) показывает текущий каталог. `cd` (change directory) меняет текущий каталог. `ls` (list) показывает содержимое каталога.

```
user@debian:~$ pwd
/home/user
user@debian:~$ ls
Документы  Загрузки  Изображения  Музыка  Шаблоны
user@debian:~$ ls -la
drwxr-xr-x  2 user user 4096 окт 23 10:00 .
drwxr-xr-x  4 root root 4096 окт 20 09:00 ..
-rw-r--r--  1 user user  220 окт 15 12:00 .bashrc
drwx------  3 user user 4096 окт 23 09:30 .config
```

Флаг `-l` даёт «длинный» формат с правами, владельцем, размером и датой. Флаг `-a` показывает скрытые файлы (начинающиеся с точки). Строка `drwxr-xr-x` — это права доступа: `d` (каталог), затем три группы по три символа для владельца, группы и остальных.

Создание и удаление

СОЗДАНИЕ СТРУКТУРЫ ПРОЕКТА — BASH

```bash
# Создаём главный каталог проекта
mkdir ~/dict-ar-ru
mkdir ~/dict-ar-ru/{scripts,data,backup,docs}

# Переходим внутрь
cd ~/dict-ar-ru

# Создаём пустые файлы
touch scripts/dict.sh scripts/config.sh
touch data/words.csv data/schema.sql

# Проверяем
ls -R
```

Команда `mkdir -p` создаёт вложенные каталоги без ошибок, если они уже есть. Фигурные скобки `{a,b,c}` — это Bash-расширение: `mkdir {scripts,data,backup}` раскрывается в три отдельных аргумента. Очень удобно для создания нескольких однотипных каталогов.

Удаление: `rmdir` удаляет пустой каталог, `rm` — файлы, `rm -r` — каталоги рекурсивно (со всем содержимым), `rm -rf` — без подтверждения. Будьте осторожны с `rm -rf`: она не спрашивает подтверждения и не отправляет в корзину.

> **ПРЕДУПРЕЖДЕНИЕ**
>
> Никогда не выполняйте `rm -rf /` или `rm -rf ~/*` — это удалит всю систему или все ваши файлы. Если нужно удалить конкретный каталог, всегда вводите полный путь и проверяйте его `ls` перед удалением.

Копирование и перемещение

Команды `cp` (copy) и `mv` (move) копируют и перемещают файлы и каталоги. `cp -r` копирует каталог рекурсивно. `mv` также используется для переименования: в Linux переименование — это перемещение файла в тот же каталог с новым именем.

КОПИРОВАНИЕ И ПЕРЕМЕЩЕНИЕ — BASH

```bash
# Копируем скрипт-шаблон
cp scripts/dict.sh scripts/dict_backup.sh

# Переименовываем
mv data/words.csv data/words_initial.csv

# Копируем весь каталог scripts в backup
cp -r scripts backup/scripts_$(date +%Y%m%d)
```

Здесь `$(date +%Y%m%d)` — это командная подстановка: результат команды `date` вставляется в строку. Получится каталог вроде `backup/scripts_20260123`. Так удобно делать ежедневные резервные копии.

Просмотр содержимого файлов

Команда `cat` (concatenate) печатает содержимое файла целиком. `less` открывает постраничный просмотрщик с поиском (нажмите `/` для поиска, `q` для выхода). `head -n 10` и `tail -n 10` показывают первые и последние 10 строк. `tail -f` следит за файлом в реальном времени — незаменимо для просмотра логов.

```
user@debian:~/dict-ar-ru$ cat scripts/config.sh
#!/bin/bash
DB_PATH="$HOME/dict-ar-ru/data/dict.db"
user@debian:~/dict-ar-ru$ head -3 data/words.csv
arabic,russian,transcription
سلام,мир,salaam
كتاب,книга,kitab
user@debian:~/dict-ar-ru$ less README.md
(постраничный просмотр)
```

Права доступа: chmod и chown

Каждый файл в Linux имеет права для трёх категорий: владельца (user), группы (group) и остальных (others). Права — это комбинация чтения (`r`, 4), записи (`w`, 2) и исполнения (`x`, 1). Сумма даёт восьмеричное число: 7 = rwx, 6 = rw-, 5 = r-x, 4 = r--.

Команда `chmod` меняет права. Чтобы сделать скрипт исполняемым, достаточно `chmod +x script.sh`. Команда `chown` меняет владельца и группу (требует sudo).

ПРАВА ДОСТУПА — BASH

```bash
# Делаем скрипт исполняемым
chmod +x scripts/dict.sh

# Эквивалентная запись в восьмеричной системе
chmod 755 scripts/dict.sh   # rwxr-xr-x

# Только владелец может читать и писать, остальные — ничего
chmod 600 data/dict.db

# Сменить владельца (требует sudo)
sudo chown user:user /var/lib/dict-ar-ru/dict.db
```

Для Bash-скриптов стандартные права — `755`: владелец может всё, остальные читать и выполнять, но не изменять. Для файлов с чувствительными данными (пароли, БД) — `600`: только владелец читает и пишет.

Практика

> **ПРАКТИКА К ГЛАВЕ 2**
>
> Создайте в домашнем каталоге структуру проекта `dict-ar-ru` с подкаталогами `scripts`, `data`, `backup`, `docs`. Внутри `scripts` создайте пустые файлы `dict.sh` и `config.sh`, сделайте их исполняемыми. В `data` создайте файл `words.csv` с заголовком `arabic,russian,transcription` и тремя строками примеров (можно с арабскими словами из примера выше). Проверьте результат командой `tree ~/dict-ar-ru` (если `tree` не установлен, установите: `sudo apt install tree`).

ГЛАВА 03 · ПЕРЕМЕННЫЕ

Переменные, строки и параметры

Переменная в Bash — это именованная ячейка памяти, хранящая строку (или, в особых случаях, число). Имя переменной может содержать буквы, цифры и знак подчёркивания, но не должно начинаться с цифры. По соглашению имена обычных переменных пишут в нижнем регистре (`db_path`), а системных и констант — в верхнем (`PATH`, `DB_NAME`).

Объявление и чтение

Переменная объявляется знаком равенства без пробелов вокруг: `name="value"`. Для чтения используется знак доллара: `$name` или `${name}`. Вторая форма безопаснее — она явно отделяет имя переменной от последующего текста.

ОБЪЯВЛЕНИЕ ПЕРЕМЕННЫХ — BASH

```bash
#!/bin/bash

WORD_AR="كتاب"
WORD_RU="книга"
DB_PATH="$HOME/dict-ar-ru/data/dict.db"

echo "Арабское: $WORD_AR"
echo "Русское: ${WORD_RU}"
echo "БД: $DB_PATH"
```

Пробелы вокруг `=` недопустимы. Запись `WORD_AR = "книга"` будет воспринята как команда `WORD_AR` с аргументами `=` и `"книга"` — и приведёт к ошибке «command not found». Это одна из самых частых ошибок новичков.

Специальные переменные

Bash имеет набор предопределённых переменных, доступных в любом скрипте. Они дают доступ к аргументам командной строки, кодам возврата, идентификатору процесса и другим системным данным.

ПЕРЕМЕННАЯ	ЗНАЧЕНИЕ
`$0`	Имя скрипта (как он был вызван)
`$1, $2, ..., $9`	Первый, второй, ..., девятый аргумент
`${10}, ${11}`	Аргументы с 10-го (нужны фигурные скобки)
`$#`	Количество аргументов
`$@`	Все аргументы как отдельные строки
`$*`	Все аргументы как одна строка
`$?`	Код возврата последней команды (0 = успех)
`$$`	PID текущего процесса (для временных файлов)
`$!`	PID последнего фонового процесса
`$HOME`, `$USER`, `$PWD`	Домашний каталог, имя пользователя, текущая директория

Различие между `$@` и `$*` тонкое. Без кавычек они ведут себя одинаково. С кавычками: `"$@"` раскрывается в отдельные строки (по одной на аргумент), а `"$*"` — в одну строку, разделённую пробелом (или первым символом `IFS`). Почти всегда нужен `"$@"`.

Кавычки: одинарные, двойные, backticks

Bash различает три вида кавычек, и их поведение важно понимать.

Одинарные кавычки `'text'` — буквальная строка. Никакие подстановки не выполняются: `'$HOME'` напечатает буквально `$HOME`, а не путь к домашнему каталогу. Используются для строк с символами `$`, которые нужно передать как есть.

Двойные кавычки `"text"` — подстановка разрешена. `"$HOME"` раскроется в `/home/user`. Это стандартный способ писать строки с переменными.

Backticks `` `cmd` `` — устаревший синтаксис командной подстановки. Современный аналог — `$(cmd)`, который предпочтительнее: легко вкладывается, читается лучше. Не используйте backticks в новых скриптах.

КАВЫЧКИ И ПОДСТАНОВКА BASH

```bash
name="Анна"

echo 'Привет, $name'        # Привет, $name
echo "Привет, $name"        # Привет, Анна
echo "Сегодня: $(date +%d.%m.%Y)"
echo "Дом: $HOME"             # Дом: /home/user
echo 'Символ $ без подстановки'
```

> **СОВЕТ**
>
> Всегда заключайте переменные в двойные кавычки, когда используете их в командах: `"$var"`, а не `$var`. Это защитит от проблем с пробелами в значениях (например, путями с пробелами) и пустыми переменными.

Командная подстановка

Синтаксис `$(command)` выполняет команду и подставляет её вывод в строку. Это позволяет динамически вычислять значения переменных. Например, получить текущую дату, количество строк в файле, имя хоста.

КОМАНДНАЯ ПОДСТАНОВКА BASH

```bash
TODAY=$(date +%Y-%m-%d)
WORD_COUNT=$(wc -l < data/words.csv)
HOST=$(hostname)
FIRST_ARABIC=$(head -1 data/words.csv | cut -d, -f1)

echo "Дата: $TODAY"
echo "Записей в словаре: $WORD_COUNT"
echo "Хост: $HOST"
```

Арифметика: $(( ))

Bash не умеет напрямую работать с числами как типом — переменные всегда строки. Но конструкция `$((expression))` выполняет целочисленные арифметику. Поддерживаются `+ - * / %` (остаток), `**` (степень), скобки для приоритета, инкремент `++` и декремент `--`.

АРИФМЕТИКА BASH

```bash
a=10
b=3

echo "$a + $b = $((a + b))"     # 13
echo "$a / $b = $((a / b))"     # 3 (целочисленное деление)
echo "$a % $b = $((a % b))"     # 1 (остаток)
echo "$a ** $b = $((a ** b))"   # 1000

# Инкремент
((a++))
echo "a теперь: $a"             # 11
```

Для дробной арифметики Bash не подходит — используйте `bc` или `awk`. Например, `echo "scale=2; 10/3" | bc` даст `3.33`.

Чтение ввода: read

Команда `read` читает строку из стандартного ввода (или из файла через перенаправление) и записывает в переменные. С флагом `-p` выводит приглашение, с `-s` — скрывает ввод (для паролей), с `-r` — не интерпретирует обратные слэши (рекомендуется всегда).

ЧТЕНИЕ ВВОДА ПОЛЬЗОВАТЕЛЯ BASH

```bash
#!/bin/bash
# Скрипт add_word.sh — добавляет слово в словарь

read -rp "Введите арабское слово: " arabic
read -rp "Введите русский перевод: " russian
read -rp "Транскрипция (латиницей): " transcription

echo "$arabic,$russian,$transcription" >> data/words.csv
echo "Добавлено: $arabic → $russian"
```

Здесь `-r` предохраняет от проблем с обратными слэшами (например, в Windows-путях), `-p` задаёт приглашение. `>>` дописывает в конец файла, не перезаписывая его.

Длина строки и срезы

Bash умеет работать со строками без вызова внешних команд. `${#var}` даёт длину строки, `${var:offset:length}` — подстроку, `${var#pattern}` и `${var%pattern}` удаляют совпадение с начала и с конца.

СТРОКОВЫЕ ОПЕРАЦИИ BASH

```bash
word=" dictionary "

echo "${#word}"              # 12 (длина с пробелами)
echo "${word:1:10}"          # dictionary (срез)
echo "${word#d}"             # ictionary (удалить d в начале)
echo "${word%d}"             # ictionary (удалить d в конце)

# Замена подстроки
echo "${word/o/0}"           # dicti0nary (первое вхождение)
echo "${word//o/0}"          # dicti0nary (все вхождения)

# Удаление пробелов в начале и конце
echo "${word## }"            # "dictionary " (только в начале)
echo "${word%% }"            # " dictionary" (только в конце)
```

Эти операции работают быстро и не порождают новых процессов. Для простых преобразований строк всегда предпочитайте их внешним утилитам.

Экспорт переменных: export

Обычная переменная видна только в текущем скрипте. Команда `export` делает её доступной дочерним процессам (командам, запущенным из этого скрипта). Это важно для конфигурации: например, переменная `DATABASE_URL` должна быть экспортирована, чтобы её видела программа-клиент БД.

ЭКСПОРТ ПЕРЕМЕННЫХ BASH

```bash
#!/bin/bash
# config.sh — общие настройки

export DB_PATH="$HOME/dict-ar-ru/data/dict.db"
export DICT_LANG_FROM="ar"
export DICT_LANG_TO="ru"
export PAGER="less"

# В другом скрипте
source config.sh   # загрузить переменные
echo "БД: $DB_PATH"
```

Команда `source` (или её синоним `.`) выполняет скрипт в текущей оболочке, не порождая новой. Так можно вынести общие настройки в отдельный файл и подключать его в разных скриптах.

Практика

> **ПРАКТИКА К ГЛАВЕ 3**
>
> Напишите скрипт `add_word.sh`, который спрашивает у пользователя арабское слово, русский перевод и транскрипцию, затем дописывает строку в CSV-файл. В конце скрипт должен выводить количество записей в файле. Сделайте скрипт исполняемым и запустите 2-3 раза с разными словами. Проверьте содержимое файла командой `cat data/words.csv`.

ГЛАВА 04 · ВВОД-ВЫВОД

Ввод-вывод и перенаправления

В Unix каждая программа имеет три стандартных потока: стандартный ввод (`stdin`, файловый дескриптор 0), стандартный вывод (`stdout`, дескриптор 1) и стандартный вывод ошибок (`stderr`, дескриптор 2). По умолчанию `stdin` подключён к клавиатуре, `stdout` и `stderr` — к экрану. Перенаправления позволяют изменить это поведение: взять ввод из файла, отправить вывод в файл или передать его другой команде.

Перенаправление вывода

Знак `>` перенаправляет `stdout` в файл, перезаписывая его. Знак `>>` дописывает в конец. Знак `<` берёт `stdin` из файла. Эти три операции — основа пакетной обработки данных в Bash.

ПЕРЕНАПРАВЛЕНИЯ — BASH

```bash
# Перезаписать файл
echo "Словарь v1.0" > docs/README.md

# Дописать
echo "Дата создания: $(date)" >> docs/README.md
echo "" >> docs/README.md
echo "## Структура" >> docs/README.md

# Взять ввод из файла
wc -l < data/words.csv
# Эквивалент: cat data/words.csv | wc -l
```

Разделение stdout и stderr

Часто нужно отделить нормальный вывод от сообщений об ошибках. Синтаксис `2>` перенаправляет только `stderr`, `2>&1` — объединяет `stderr` с `stdout`. Так можно записать ошибки в отдельный лог или проигнорировать их.

РАЗДЕЛЕНИЕ ПОТОКОВ BASH

```bash
# Только stdout в файл, ошибки на экран
sqlite3 dict.db "SELECT * FROM words" > data/dump.txt

# Только stderr в файл
sqlite3 dict.db "SELECT * FROM words" 2> logs/error.log

# Объединить stdout и stderr в один файл
sqlite3 dict.db ".schema" > logs/schema.log 2>&1

# Игнорировать ошибки (отправить в /dev/null)
rm -f /tmp/nonexistent 2>/dev/null
```

Порядок перенаправлений важен: `>file 2>&1` сначала открывает `file` для stdout, затем дублирует stderr в текущий stdout (то есть в файл). Обратный порядок `2>&1 >file` даст другой результат: stderr пойдёт на экран, а stdout — в файл.

Конвейер (pipe)

Вертикальная черта `|` — конвейер — передаёт stdout одной команды на stdin другой. Это одна из самых мощных идей Unix: маленькие утилиты комбинируются в сложные pipeline. Конвейер создаёт неявный канал между процессами: данные проходят напрямую, без промежуточных файлов.

```
user@debian:~/dict-ar-ru$ cat data/words.csv | head -5 | tail -3
كتاب,книга,kitab
باب,дверь,bab
ماء,вода,maa

user@debian:~/dict-ar-ru$ cat data/words.csv | grep "كتاب"
كتاب,книга,kitab

user@debian:~/dict-ar-ru$ cat data/words.csv | cut -d, -f2 | sort | uniq -c |
sort -rn | head
      3 книга
      2 дверь
      1 вода
```

Последний конвейер показывает типичный анализ данных: берём второй столбец (русские слова), сортируем, считаем дубликаты, сортируем по убыванию, оставляем топ-10. Каждая команда простая, вместе они дают мощный аналитический инструмент.

tee: Т-образный разветвитель

Команда `tee` читает stdin и пишет его одновременно в файл и в stdout. Это позволяет «подсмотреть» за данными в середине конвейера, не разрывая его. Полезно для логирования.

TEE — ЛОГИРОВАНИЕ КОНВЕЙЕРА BASH

```bash
# Сохранить вывод в файл И показать на экране
sqlite3 dict.db "SELECT * FROM words" | tee data/dump.txt | head -10

# С флагом -a — дописывать, а не перезаписывать
echo "Запуск скрипта: $(date)" | tee -a logs/script.log

# Сохранить и stdout, и stderr в один лог, но видеть только stdout
some_command 2>&1 | tee logs/all.log | grep -v "^[Ee]rror"
```

xargs: из stdin в аргументы

Конвейер передаёт данные через stdin, но некоторые команды ожидают аргументы, а не stdin. Например, `rm` хочет имена файлов как аргументы. `xargs` берёт строки из stdin и превращает их в аргументы для указанной команды.

XARGS BASH

```bash
# Найти все .bak файлы и удалить
find . -name "*.bak" | xargs rm

# Безопаснее: имена с пробелами и спецсимволами
find . -name "*.bak" -print0 | xargs -0 rm

# Запустить sqlite3 с SQL из файла
echo "schema.sql" | xargs -I{} sqlite3 dict.db < {}

# Пакетная обработка: для каждого слова в файле
cat arabic_words.txt | xargs -I{} ./translate.sh {}
```

Флаг `-0` в связке с `find -print0` — стандартный способ безопасно передавать имена файлов с пробелами, кавычками и переводами строк. Всегда используйте эту пару для надёжных скриптов.

Here-doc и here-string

Here-doc (встроенный документ) позволяет передать многострочный текст в команду без создания файла. Синтаксис: `command <<DELIMITER ... DELIMITER`. Строки между

ограничителями становятся stdin команды. Здесь DELIMITER — произвольное слово, обычно `EOF` или `SQL`.

HERE-DOC ДЛЯ SQL — BASH

```bash
#!/bin/bash
# apply_schema.sh — применить схему к SQLite

sqlite3 "$DB_PATH" <<SQL
CREATE TABLE IF NOT EXISTS words (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    arabic TEXT NOT NULL,
    russian TEXT NOT NULL,
    transcription TEXT,
    created_at TEXT DEFAULT (datetime('now'))
);
CREATE INDEX IF NOT EXISTS idx_arabic ON words(arabic);
CREATE INDEX IF NOT EXISTS idx_russian ON words(russian);
SQL

echo "Схема применена"
```

Если ограничитель написать без кавычек (`<<SQL`), внутри here-doc работают подстановки переменных. Если в кавычках (`<<'SQL'`) — текст передаётся буквально, без подстановок. Для SQL с переменными Bash используйте первый вариант, для SQL с символом `$` — второй.

Here-string — это «одноразовый» here-doc для одной строки: `command <<<"text"`. Удобно для коротких данных.

HERE-STRING — BASH

```bash
# Передать одну строку в bc для вычисления
bc <<< "2 + 2"

# Поиск в строке
grep "книга" <<< "это книга про bash"

# JSON в Python
python3 -c "import sys, json; print(json.loads(sys.stdin.read()))" <<< '{"a": 1}'
```

Практика

> **ПРАКТИКА К ГЛАВЕ 4**
>
> Напишите скрипт `stats.sh`, который читает файл `data/words.csv` и выводит: общее количество записей (без заголовка), количество уникальных русских переводов, самый длинный арабский слово. Используйте конвейеры из `cat`, `tail`, `cut`, `sort`, `uniq`, `awk`. Результат сохраните в файл `data/stats.txt` с помощью `tee`, а на экран покажите только первые 5 строк.

ГЛАВА 05 · УСЛОВИЯ

Условия и ветвление

Условные операторы позволяют скрипту принимать решения в зависимости от данных. В Bash есть два основных способа: `if-elif-else-fi` для сложных проверок и `case-esac` для выбора из нескольких альтернатив. Оба опираются на код возврата команды: 0 означает успех (истина), ненулевое значение — ошибку (ложь).

Команда test и [[ ]]

Команда `test` (или её синоним `[ ]`) проверяет условия и возвращает 0 или 1. Современный Bash предлагает улучшенный синтаксис `[[ ]]`, который поддерживает регулярные выражения, более безопасные операции со строками и не требует кавычек вокруг переменных в большинстве случаев.

ПРОВЕРКА	ИСТИННА, ЕСЛИ
`-z "$str"`	Строка пустая
`-n "$str"`	Строка непустая
`"$a" = "$b"`	Строки равны
`"$a" ≠ "$b"`	Строки не равны
`"$a" < "$b"`	*а*лексикографическираньшеb (в [[ ]])
`-eq`, `-ne`, `-lt`, `-le`, `-gt`, `-ge`	Числовые сравнения: равно, не равно, меньше, меньше или равно, больше, больше или равно
`-e "$f"`	Файл существует
`-f "$f"`	Обычный файл
`-d "$d"`	Каталог
`-r "$f"`, `-w "$f"`, `-x "$f"`	Файл доступен для чтения / записи / исполнения
`-s "$f"`	Файл существует и непустой

Синтаксис if

IF-ELIF-ELSE BASH

```bash
#!/bin/bash
# check_args.sh — проверка аргументов

if [[ $# -eq 0 ]]; then
    echo "Использование: $0 <arabic> <russian>"
    echo "Пример: $0 كتاب книга"
    exit 1
elif [[ $# -eq 1 ]]; then
    echo "Ошибка: не указан русский перевод"
    exit 1
else
    echo "Добавляем: $1 → $2"
fi
```

Ключевое слово `then` можно писать на той же строке после `;` или на следующей. `fi` (if наоборот) закрывает блок. Аналогично работают `case ... esac`, `for ... done`, `while ... done`.

Логические операторы

Bash предлагает несколько способов комбинировать условия. Внутри `[[ ]]` работают `&&` (И), `||` (ИЛИ), `!` (НЕ). Между командами `&&` и `||` работают как «выполни следующее, если предыдущая успешна/неуспешна».

ЛОГИЧЕСКИЕ ОПЕРАТОРЫ BASH

```bash
# Внутри [[ ]]
if [[ -n "$word" && "$word" != "quit" ]]; then
    echo "Слово: $word"
fi

# Между командами
[[ -f data/dict.db ]] && echo "БД существует"
[[ -f data/dict.db ]] || echo "БД не найдена"

# Эквивалентно if-then
[[ -f data/dict.db ]] && {
    echo "БД найдена"
    sqlite3 data/dict.db ".tables"
}
```

case: выбор по шаблону

Когда нужно сравнить значение с несколькими вариантами, `case` удобнее цепочки `if-elif`. Шаблоны поддерживают glob-символы: `*` (любая строка), `?` (один символ), `[abc]` (один из перечисленных), `|` (или).

CASE ДЛЯ ПОДКОМАНД — BASH

```bash
#!/bin/bash
# dict.sh — главная команда словаря

ACTION="$1"
shift

case "$ACTION" in
    add)
        echo "Добавление слова: $@"
        ./add_word.sh "$@"
        ;;
    search)
        echo "Поиск: $1"
        ./search_word.sh "$1"
        ;;
    list|ls)
        echo "Все слова:"
        ./list_words.sh
        ;;
    stat*)
        echo "Статистика"
        ./stats.sh
        ;;
    help|--help|-h)
        echo "Использование: dict.sh {add|search|list|stats|help}"
        ;;
    *)
        echo "Неизвестная команда: $ACTION"
        echo "Введите 'dict.sh help' для справки"
        exit 1
        ;;
esac
```

Здесь `shift` убирает первый аргумент (`$1`), сдвигая все остальные влево: `$2` становится `$1` и так далее. После `shift` мы получаем «чистые» аргументы подкоманды в `$@`.

Шаблон `list|ls)` срабатывает на оба варианта. `stat*)` — на любое слово, начинающееся с `stat` (stats, statistics). `*)` в конце — это ветка по умолчанию,

срабатывает, если ни один шаблон не подошёл.

Проверка существования файлов

Один из самых частых сценариев — проверить, существует ли файл, и только потом с ним работать. Это делает скрипты устойчивыми.

ЗАЩИТА ОТ ОТСУТСТВУЮЩИХ ФАЙЛОВ BASH

```bash
DB="data/dict.db"

if [[ ! -f "$DB" ]]; then
    echo "БД не найдена, создаю новую..."
    sqlite3 "$DB" <<SQL
CREATE TABLE words (id INTEGER PRIMARY KEY, arabic TEXT, russian TEXT);
SQL
fi

# То же в одну строку
[[ -f "$DB" ]] || { echo "Создаю $DB"; touch "$DB"; }
```

Сравнение строк с учётом локали

Сравнение строк зависит от текущей локали. Для русского и арабского языков важно установить правильную локаль через `LC_COLLATE`, иначе порядок сортировки будет некорректным.

ЛОКАЛЬ И СРАВНЕНИЕ BASH

```bash
# Проверить текущую локаль
locale

# Установить UTF-8 локаль (если не установлена)
sudo dpkg-reconfigure locales
# Выбрать ru_RU.UTF-8 и en_US.UTF-8

# В скрипте принудительно
export LC_ALL=ru_RU.UTF-8
export LANG=ru_RU.UTF-8

# Сравнение кириллических строк
if [[ "книга" < "ручка" ]]; then
    echo "книга идёт раньше"
fi
```

Практика

> **ПРАКТИКА К ГЛАВЕ 5**
>
> Напишите главный скрипт `dict.sh` с подкомандами `add`, `search`, `list`, `help` на основе `case`. Подкоманда `add` должна вызывать `add_word.sh` из главы 3, `list` — выводить содержимое CSV, `search` — искать слово через `grep`. Если подкоманда не указана или неизвестна, выводите справку. Проверьте все ветки.

ГЛАВА 06 · ЦИКЛЫ

Циклы

Циклы позволяют выполнять блок кода многократно. В Bash есть три основных вида циклов: `for` для перебора списка, `while` для повторения, пока условие истинно, и `until` — пока ложно. Все они закрываются ключевым словом `done`.

Цикл for

Цикл `for` перебирает элементы списка. Список может быть задан явно, результатом раскрытия шаблона (glob), или выводом команды в `$()`.

FOR — РАЗНЫЕ ВИДЫ СПИСКОВ — BASH

```bash
# Явный список
for word in книга ручка стол окно; do
    echo "Слово: $word"
done

# Перебор файлов по шаблону
for f in data/*.csv; do
    echo "Файл: $f ($(wc -l < "$f") строк)"
done

# Перебор аргументов скрипта
for arg in "$@"; do
    echo "Аргумент: $arg"
done

# Перебор вывода команды
for user in $(cut -d: -f1 /etc/passwd); do
    echo "Пользователь: $user"
done

# C-style: числовой диапазон
for ((i=1; i<=10; i++)); do
    echo "Число: $i"
done
```

Цикл в C-стиле `for ((init; cond; step))` удобен, когда нужен числовой счётчик. Двойные скобки обязательны. Диапазоны можно задавать и так: `for i in {1..10}` или `for i in {1..10..2}` (с шагом 2).

Цикл while

`while` выполняет блок, пока команда (или условие `[[ ]]`) возвращает 0. Это классический способ построчного чтения файла.

WHILE — ПОСТРОЧНОЕ ЧТЕНИЕ — BASH

```bash
# Читать файл построчно
while IFS= read -r line; do
    echo "Строка: $line"
done < data/words.csv

# Читать CSV, разбивая по запятой
while IFS=, read -r arabic russian transcription; do
    echo "$arabic → $russian ($transcription)"
done < <(tail -n +2 data/words.csv)

# Бесконечный цикл с выходом по условию
while true; do
    read -rp "Введите слово (или 'quit'): " word
    [[ "$word" == "quit" ]] && break
    echo "Вы ввели: $word"
done
```

Конструкция `IFS=, read -r a b c` читает строку, разделяя её по запятой на три переменные. `IFS=` (пустой) перед `read` без аргумента сохраняет начальные и конечные пробелы. Подстановка процесса `<(<cmd>)` передаёт вывод команды как файл — это лучше, чем конвейер, потому что переменные из `while` остаются видимыми после цикла.

break и continue

`break` выходит из цикла досрочно, `continue` переходит к следующей итерации. С числовым аргументом: `break 2` выходит из двух уровней вложенности.

BREAK И CONTINUE BASH

```bash
# Пропустить пустые строки, выйти на слове "stop"
while IFS= read -r line; do
    [[ -z "$line" ]] && continue   # пустая строка
    [[ "$line" == "stop" ]] && break
    echo "$line"
done < input.txt

# continue с числом — перейти к следующей итерации внешнего цикла
for i in {1..5}; do
    for j in {1..5}; do
        (( (i*j) % 2 == 0 )) && continue 2
        echo -n "$i*$j=$((i*j)) "
    done
    echo
done
```

Цикл until

`until` — это `while` с обратным условием: выполняется, пока условие ложно. Используется реже, но иногда семантически правильнее.

UNTIL — ЖДАТЬ ПОЯВЛЕНИЯ ФАЙЛА BASH

```bash
# Ждать, пока файл не появится (с таймаутом)
tries=0
until [[ -f /tmp/ready.txt ]] || (( tries >= 60 )); do
    sleep 1
    ((tries++))
done

[[ -f /tmp/ready.txt ]] && echo "Файл появился" || echo "Таймаут"
```

select: интерактивное меню

Команда `select` создаёт интерактивное меню: печатает пронумерованный список, ждёт ввода пользователя, помещает выбранный элемент в переменную. Удобно для простых CLI-приложений.

SELECT — МЕНЮ BASH

```bash
#!/bin/bash
# menu.sh — меню словаря

PS3="Выберите действие: "
select action in "Добавить слово" "Найти слово" "Показать все" "Выход"; do
    case "$action" in
        "Добавить слово")   ./add_word.sh ;;
        "Найти слово")     read -rp "Слово: " w; ./search_word.sh "$w" ;;
        "Показать все")     ./list_words.sh ;;
        "Выход")           break ;;
        *)                 echo "Неверный выбор" ;;
    esac
done
```

Переменная `PS3` задаёт текст приглашения. После выбора срабатывает `case`, и меню показывается снова, пока не выполнится `break`.

Практика

> **ПРАКТИКА К ГЛАВЕ 6**
>
> Напишите скрипт `import_csv.sh`, который читает CSV-файл `data/words.csv` построчно, пропускает заголовок, и для каждой строки добавляет запись в текстовый лог `logs/import.log` в формате `[дата] добавлено: arabic → russian`. В конце выводите количество обработанных строк. Используйте `while read` с разделителем `,`.

ГЛАВА 07 · ФУНКЦИИ

Функции

Функции в Bash — это именованные блоки кода, которые можно вызывать многократно. Они принимают позиционные параметры (`$1`, `$2` внутри функции — это аргументы функции, а не скрипта), возвращают код завершения через `return` и могут использовать локальные переменные через `local`. Функции — основной способ структурирования больших скриптов.

Объявление и вызов

Есть два синтаксиса объявления функции. `function name() { ... }` и `name() { ... }` — эквивалентны, второй короче и портативнее. Функция должна быть объявлена до её вызова — Bash читает скрипт сверху вниз.

ОБЪЯВЛЕНИЕ ФУНКЦИИ — BASH

```bash
#!/bin/bash

# Объявление
greet() {
    echo "Привет, $1!"
}

# Альтернативный синтаксис с ключевым словом function
function add_word() {
    echo "$1,$2,$3" >> data/words.csv
}

# Вызов
greet "Мир"                # Привет, Мир!
add_word "كتاب" "книга" "kitab"
```

Параметры и return

Внутри функции `$1 .. $N` — это аргументы функции, `$0` — имя функции (в Bash 4+), `$#` — количество аргументов функции, `$@` — все аргументы функции. Переменные `$?`, `$$`, `$!` относятся к процессу в целом.

Команда `return` выходит из функции с указанным кодом возврата (0-255). Без аргумента возвращает код последней команды. Код возврата проверяется через `$?` сразу после вызова. `return` не возвращает значение — для этого используют подстановку команды.

ПАРАМЕТРЫ И RETURN BASH

```bash
check_word() {
    if [[ -z "$1" ]]; then
        echo "Ошибка: пустое слово" >&2
        return 1
    fi
    return 0
}

# Вызов и проверка кода
if check_word "книга"; then
    echo "Слово валидно"
else
    echo "Отклонено"
fi
```

Возврат значения через подстановку

Чтобы функция вернула строку, её выводят через `echo` и захватывают в вызывающем коде через `$()`. Это идиоматичный способ возврата значений в Bash.

ВОЗВРАТ ЗНАЧЕНИЯ BASH

```bash
normalize_arabic() {
    # Удалить огласовки и нормализовать алифы
    echo "$1" | sed 's/[\x{064B}-\x{0652}]//g; s/ا/آ/g; s/ا/أ/g; s/ا/إ/g'
}

format_entry() {
    local ar="$1"
    local ru="$2"
    local tr="$3"
    echo "$ar | $ru | $tr"
}

# Использование
clean=$(normalize_arabic "كِتَابٌ")
echo "Нормализованное: $clean"      # كتاب

entry=$(format_entry "سلام" "мир" "salaam")
echo "$entry"                       # سلام | мир | salaam
```

Локальные переменные: local

Без `local` переменные внутри функции изменяют глобальные. Это частый источник багов. `local` создаёт переменную, видимую только внутри функции и её дочерних функций.

LOCAL — ИЗОЛЯЦИЯ ПЕРЕМЕННЫХ BASH

```bash
counter=0   # глобальная

increment() {
    local step="${1:-1}"   # локальная, по умолчанию 1
    ((counter += step))
    echo "Шаг: $step, счётчик: $counter"
}

increment 5      # Шаг: 5, счётчик: 5
increment 3      # Шаг: 3, счётчик: 8
echo "$step"          # (пусто) — step не видна снаружи
echo "$counter"        # 8
```

Конструкция `${1:-1}` — это значение первого аргумента, или `1`, если аргумент пустой. Это стандартный приём для задания значений по умолчанию.

Рекурсия

Bash поддерживает рекурсию — функция может вызывать сама себя. Это удобно для задач вроде обхода дерева каталогов, факториала, вычисления Фибоначчи. Но помните о лимите вложенности (обычно около 1000 уровней).

РЕКУРСИЯ — ФАКТОРИАЛ BASH

```bash
factorial() {
    local n="$1"
    if (( n ⩽ 1 )); then
        echo 1
    else
        local sub=$(factorial $((n - 1)))
        echo $((n * sub))
    fi
}

echo "5! = $(factorial 5)"   # 120
echo "10! = $(factorial 10)"  # 3628800
```

Библиотека функций

Функции можно вынести в отдельный файл и подключать через `source`. Это позволяет переиспользовать код в нескольких скриптах.

LIB/DICT_FUNCTIONS.SH — BASH

```bash
#!/bin/bash
# Библиотека функций для словаря

dict_add() {
    local ar="$1" ru="$2" tr="$3"
    [[ -z "$ar" || -z "$ru" ]] && return 1
    sqlite3 "$DB_PATH" "INSERT INTO words (arabic, russian, transcription) VALUES
('$ar', '$ru', '$tr');"
}

dict_search_ar() {
    sqlite3 "$DB_PATH" "SELECT * FROM words WHERE arabic LIKE '%$1%';"
}

dict_search_ru() {
    sqlite3 "$DB_PATH" "SELECT * FROM words WHERE russian LIKE '%$1%';"
}

dict_count() {
    sqlite3 "$DB_PATH" "SELECT COUNT(*) FROM words;"
}

dict_list() {
    sqlite3 -header -column "$DB_PATH" "SELECT * FROM words ORDER BY arabic;"
}
```

ИСПОЛЬЗОВАНИЕ БИБЛИОТЕКИ — BASH

```bash
#!/bin/bash
# dict.sh — главный скрипт

source "$(dirname "$0")/lib/dict_functions.sh"
source "$(dirname "$0")/config.sh"

case "$1" in
    add)     dict_add "$2" "$3" "$4" ;;
    search)  dict_search_ar "$2" ;;
    count)   dict_count ;;
    list)    dict_list ;;
esac
```

`$(dirname "$0")` даёт каталог, в котором лежит скрипт. Так можно запускать его из любого места, и пути будут корректными.

Практика

> **ПРАКТИКА К ГЛАВЕ 7**
>
> Создайте библиотеку `lib/dict_functions.sh` с функциями `dict_add`, `dict_search`, `dict_count`, `dict_list`, работающими с CSV-файлом (не с БД). Используйте `local` для всех временных переменных. Напишите главный скрипт `dict.sh`, который подключает библиотеку и маршрутизирует подкоманды. Проверьте: добавьте 3 слова, найдите одно по подстроке, выведите список.

ГЛАВА 08 · ТЕКСТОВЫЕ УТИЛИТЫ

Текстовые утилиты: grep, sed, awk

Три классические утилиты Unix — `grep`, `sed` и `awk` — образуют мощный инструментарий для обработки текста. `grep` ищет по шаблону, `sed` редактирует поток, `awk` выполняет программы над структурированным текстом. Владение этими утилитами позволяет решать задачи обработки словаря в одну-две команды вместо десятков строк на Bash.

grep: поиск по шаблону

`grep` (Global Regular Expression Print) печатает строки, содержащие совпадение с регулярным выражением. Это самый частый инструмент поиска в Unix.

ФЛАГ	НАЗНАЧЕНИЕ
`-i`	Игнорировать регистр
`-v`	Инвертировать: печатать строки БЕЗ совпадения
`-n`	Печатать номер строки
`-c`	Только количество совпадающих строк
`-r`	Рекурсивно по каталогу
`-l`	Только имена файлов с совпадениями
`-E`	Расширенные регулярные выражения (как egrep)
`-F`	Фиксированные строки (не regex)
`-o`	Печатать только совпадение, не всю строку
`-w`	Только целые слова

GREP — ПОИСК В СЛОВАРЕ BASH

```bash
# Найти все строки со словом "книга"
grep "книга" data/words.csv

# Без учёта регистра, с номерами строк
grep -in "КНИГА" data/words.csv

# Найти арабское слово (точное совпадение первой колонки)
grep -E "^كتاب," data/words.csv

# Подсчитать количество слов на букву "к"
grep -cE ",к" data/words.csv

# Рекурсивный поиск по всем .csv в каталоге
grep -rl "سلام" data/

# Инвертированный поиск: все строки без транскрипции
grep -vE ",[a-z]+$" data/words.csv
```

sed: потоковый редактор

`sed` (Stream Editor) применяет команды редактирования к каждой строке входного потока. Самая частая команда — `s/old/new/g`: заменить `old` на `new` глобально.

SED — ЗАМЕНЫ BASH

```bash
# Заменить все запятые на табуляции (CSV → TSV)
sed 's/,/\t/g' data/words.csv

# Заменить только первое вхождение в строке
sed 's/,/|/' data/words.csv

# Удалить строки, содержащие "TODO"
sed '/TODO/d' notes.txt

# Удалить пустые строки
sed '/^$/d' data/words.csv

# Печатать только строки с 5 по 10
sed -n '5,10p' data/words.csv

# Удалить огласовки из арабского текста
sed 's/[\x{064B}-\x{0652}]//g' input.txt

# Редактировать файл на месте (с бэкапом .bak)
sed -i.bak 's/old/new/g' file.txt
```

Флаг `-i` редактирует файл на месте (без вывода в stdout). С суффиксом `-i.bak` создаёт резервную копию. Без суффикса `-i` — без бэкапа. Будьте осторожны: всегда сначала проверяйте команду без `-i`, потом добавляйте его.

Флаг `-E` включает расширенные регулярные выражения (как `grep -E`). В них `+`, `?`, `{n,m}`, `|`, `()` работают без обратного слэша.

awk: язык обработки структурированного текста

`awk` — это полноценный язык программирования, оптимизированный для обработки табличных данных. Он читает файл построчно, разбивает на поля по разделителю (по умолчанию пробел), и применяет программу вида `условие { действие }`.

AWK — ОСНОВЫ — BASH

```bash
# Печатать первую колонку каждого файла CSV
awk -F, '{print $1}' data/words.csv

# Печатать вторую колонку только для строк с "книга"
awk -F, '/книга/ {print $2}' data/words.csv

# Пропустить заголовок, посчитать строки
awk -F, 'NR>1 {count++} END {print count}' data/words.csv

# Поменять колонки местами
awk -F, '{print $2","$1","$3}' data/words.csv > data/reversed.csv

# Условие по длине слова в первой колонке
awk -F, 'length($1) > 4 {print $0}' data/words.csv

# Сумма длин русских слов (колонка 2)
awk -F, 'NR>1 {sum += length($2)} END {print "Средняя длина:", sum/(NR-1)}'
data/words.csv
```

Встроенные переменные awk: `NR` (номер строки), `NF` (количество полей), `$0` (вся строка), `$1`..`$N` (поля), `FS` (разделитель полей), `OFS` (разделитель в выводе). Блок `BEGIN` выполняется до чтения, `END` — после.

AWK — ОТЧЁТЫ BASH

```bash
# Простой отчёт по словарю
awk -F, '
BEGIN {
    print "═══ Отчёт по словарю ═══"
    print "Слово\t\tПеревод\t\tТранскрипция"
    print "-----\t\t-------\t\t------------"
}
NR>1 {
    printf "%-15s\t%-15s\t%s\n", $1, $2, $3
    count++
}
END {
    print "-----"
    print "Всего записей:", count
}
' data/words.csv
```

Другие полезные утилиты

CUT, SORT, UNIQ, TR, PASTE BASH

```bash
# cut — вырезать колонки
cut -d, -f1,2 data/words.csv     # 1-я и 2-я колонки
cut -c1-5 file.txt               # символы 1-5 каждой строки

# sort — сортировка
sort data/words.csv               # по умолчанию
sort -t, -k2 data/words.csv       # по 2-й колонке
sort -u data/words.csv             # уникальные строки
sort -r -n numbers.txt             # числовая, по убыванию

# uniq — удалить повторяющиеся строки (только соседние)
sort file.txt | uniq               # всегда парить с sort
sort file.txt | uniq -c           # с количеством повторов
sort file.txt | uniq -d           # только дубликаты

# tr — замена/удаление символов
echo "Hello" | tr 'a-z' 'A-Z'     # HELLO
echo "a,b,c" | tr ',' '\n'     # a (newline) b (newline) c
echo "Hello 123" | tr -d '0-9'  # Hello

# paste — объединить файлы построчно
paste arabic.txt russian.txt > combined.txt
```

Практика

> **ПРАКТИКА К ГЛАВЕ 8**
>
> Создайте файл `data/words.csv` минимум с 10 записями. Напишите скрипт `analyze.sh`, который: (1) считает общее количество слов; (2) находит самое длинное арабское слово и его перевод; (3) выводит топ-3 самых частых русских перевода; (4) находит дубликаты по арабскому полю. Используйте `awk`, `sort`, `uniq`, `grep`.

ГЛАВА 09 · REGEX

Регулярные выражения

Регулярные выражения (regex) — это компактный язык для описания шаблонов в тексте. Они применяются в `grep`, `sed`, `awk`, `bash [[ ]]=~` и большинстве языков программирования. В контексте словаря regex позволяет валидировать арабские и русские слова, искать по морфемам, нормализовать написание.

Базовый синтаксис

СИМВОЛ	ЗНАЧЕНИЕ
`.`	Любой символ (кроме перевода строки)
`^`	Начало строки
`$`	Конец строки
`*`	0 или более повторений предыдущего
`+`	1 или более (только в ERE)
`?`	0 или 1 (только в ERE)
`{n}`, `{n,m}`	n повторений / от n до m
`[abc]`	Любой из перечисленных
`[^abc]`	Любой, кроме перечисленных
`[a-z]`	Диапазон (только ASCII)
`(abc|def)`	Группа с альтернативой (ERE)
`\d`, `\w`, `\s`	Только в Perl-regex (grep -P). В POSIX: классы

В Bash есть три диалекта регулярных выражений: BRE (Basic, по умолчанию в grep и sed), ERE (Extended, флаг `-E`), PCRE (Perl, флаг `-P` в GNU grep). Для большинства задач достаточно ERE — его и будем использовать.

Классы POSIX

Поскольку `[a-z]` работает только для ASCII, для Unicode (русский, арабский) используют классы POSIX или диапазоны Unicode.

КЛАСС	ОПИСАНИЕ
`[[:alpha:]]`	Буквы (включая Unicode при правильной локали)
`[[:digit:]]`	Цифры 0-9
`[[:alnum:]]`	Буквы и цифры
`[[:space:]]`	Пробельные символы
`[[:upper:]]`, `[[:lower:]]`	Верхний/нижний регистр
`[[:punct:]]`	Знаки препинания

Валидация арабских и русских слов

Арабский алфавит в Unicode занимает диапазон U+0600 — U+06FF (Basic Arabic) и U+0750 — U+077F (Arabic Supplement). Кириллица — U+0400 — U+04FF.

ВАЛИДАЦИЯ СЛОВ BASH

```bash
#!/bin/bash
# validate_word.sh — проверка слова

validate_arabic() {
    local word="$1"
    # Только арабские буквы (диапазон 0600-06FF), без огласовок
    if [[ "$word" =~ ^[\u0600-\u06FF]+$ ]]; then
        return 0
    else
        return 1
    fi
}

validate_russian() {
    local word="$1"
    # Кириллица + дефис (для сложных слов)
    if [[ "$word" =~ ^[а-яА-ЯёЁ-]+$ ]]; then
        return 0
    else
        return 1
    fi
}

# Использование
if validate_arabic "كتاب"; then echo "OK"; fi
if validate_russian "привет"; then echo "OK"; fi
if ! validate_russian "hello"; then echo "Не русское"; fi
```

Поиск по шаблону в grep

GREP -E — ПОИСК BASH

```bash
# Слова, начинающиеся с определённой буквы
grep -E "^ل" data/words.csv        # арабские на ل
grep -E ",к" data/words.csv         # русские на к (после запятой)

# Слова из 4 букв
grep -E "^.{4}," data/words.csv

# Слова длиннее 5 букв
grep -E "^.{6,}," data/words.csv

# Транскрипция, начинающаяся с гласной
grep -E ",[aeiou][a-z]*$" data/words.csv

# Дубликаты (одинаковые арабские слова)
cut -d, -f1 data/words.csv | sort | uniq -d
```

Замены в sed с regex

SED -E — ЗАМЕНЫ BASH

```bash
# Удалить все не-буквенные символы из арабского поля
sed -E 's/^([^,]+)[^a-zA-Za-яA-ЯёЁ\u0600-\u06FF,]+,/\1,/' data/words.csv

# Привести к нижнему регистру (через tr, не sed)
echo "Привет" | tr 'А-ZA-Я' 'a-za-я'

# Нормализовать алифы: ا → إ ,أ ,آ
sed -E 's/[إأآ]/ا/g' input.txt

# Удалить та марбута (ة) → ه (для нечёткого поиска)
sed 's/ه/$ة/g' input.txt

# Обернуть группу в захват
echo "книга" | sed -E 's/^(.)(.*)$/\U\1\E\2/'  # Книга (заглавная первая)
```

Bash [[ ]]=~ и захват групп

Оператор `=~` внутри `[[ ]]` проверяет совпадение с расширенным регулярным выражением. Захваченные группы доступны в массиве `${BASH_REMATCH[@]}`.

BASH_REMATCH BASH

```bash
parse_entry() {
    local line="$1"
    if [[ "$line" =~ ^([^,]+),([^,]+),([^,]+)$ ]]; then
        echo "Арабское: ${BASH_REMATCH[1]}"
        echo "Русское:  ${BASH_REMATCH[2]}"
        echo "Транскр.: ${BASH_REMATCH[3]}"
    else
        echo "Неверный формат" >&2
        return 1
    fi
}

parse_entry "كتاب,книга,kitab"
```

СОВЕТ

Никогда не заключайте regex в кавычки внутри `[[ ]]=~` — Bash раскроет их как строку, а не как regex. Используйте переменную: `re='^[a-z]+$'; [[ "$s" =~ $re ]]`. Это позволяет менять regex динамически и избегает проблем с экранированием.

Практика

ПРАКТИКА К ГЛАВЕ 9

Расширьте скрипт `add_word.sh` из главы 3 валидацией: арабское слово должно содержать только арабские буквы (без цифр и пробелов), русское — только кириллицу и дефис, транскрипция — только латиницу. При невалидном вводе выводите ошибку и просите ввести заново. Используйте `[[ ]]=~` и функции-валидаторы.

ГЛАВА 10 · АРАБСКИЙ

Арабские шрифты и ввод в Debian

Арабское письмо — одно из самых распространённых в мире: им пользуются более 400 миллионов человек. Оно пишется справа налево (RTL), буквы соединяются, форма буквы зависит от позиции (изолированная, начальная, срединная, конечная), а над и под буквами могут стоять огласовки — диакритические знаки, обозначающие гласные. Для корректной работы с арабским в Debian нужно установить шрифты, настроить раскладку клавиатуры и убедиться, что терминал правильно отображает RTL-текст.

Установка арабских шрифтов

Debian включает несколько пакетов шрифтов с поддержкой арабского. Главные — `fonts-noto-core` (Noto Sans Arabic, современный шрифт от Google) и `fonts-arabeyes` (классические шрифты от проекта ArabEyes). Для более полной коллекции есть `fonts-noto-extra` и `fonts-hosny-amiri` (Amiri — каллиграфический шрифт, имитирующий насх).

```
user@debian:~$ sudo apt install -y fonts-noto-core fonts-noto-extra fonts-
arabeyes fonts-hosny-amiri fonts-kacst
Чтение списков пакетов... Готово
...
Настраивается пакет fonts-noto-core (20201225-1) ...
Настраивается пакет fonts-hosny-amiri (0.111-3) ...

user@debian:~$ fc-list | grep -i arabic | head
/usr/share/fonts/truetype/noto/NotoSansArabic-Regular.ttf: Noto Sans
Arabic:style=Regular
/usr/share/fonts/truetype/noto/NotoNaskhArabic-Regular.ttf: Noto Naskh
Arabic:style=Regular
/usr/share/fonts/truetype/amiri/Amiri-Regular.ttf: Amiri:style=Regular
/usr/share/fonts/truetype/kacst/KacstOne.ttf: KacstOne:style=Regular
```

Команда `fc-list` (от fontconfig) показывает все установленные шрифты. Фильтр `grep -i arabic` оставляет только те, что поддерживают арабский. Если список пустой, шрифты не установились или не подхватились — выполните `fc-cache -fv` для пересборки кэша.

Настройка раскладки клавиатуры

Для ввода арабского текста нужна вторая раскладка. В Debian раскладки настраиваются в файле `/etc/default/keyboard` или через графический интерфейс GNOME.

/ETC/DEFAULT/KEYBOARD CONFIG

```
# /etc/default/keyboard
XKBMODEL="pc105"
XKBLAYOUT="us,ru,ar"          # английская, русская, арабская
XKBVARIANT=",,"                 # варианты (пусто = default)
XKBOPTIONS="grp:alt_shift_toggle,grp_led:scroll"
# grp:alt_shift_toggle — переключение по Alt+Shift
# grp_led:scroll — индикация лампочкой ScrollLock
```

После изменения файла примените настройки: `sudo setupcon` (для текстовой консоли) или перезайдите в X-сессию. В GNOME те же настройки доступны через «Настройки → Клавиатура + Регион и язык».

Временно переключить раскладку из терминала (для X-сессии):

```
user@debian:~$ setxkbmap -layout us,ru,ar -option grp:alt_shift_toggle
user@debian:~$ setxkbmap -query
rules:      evdev
model:      pc105
layout:     us,ru,ar
options:    grp:alt_shift_toggle

user@debian:~$ setxkbmap ar   # временно только арабская
user@debian:~$ setxkbmap us,ru,ar -option grp:alt_shift_toggle  # вернуть
```

Ввод арабского в терминале

Современные эмуляторы терминала (GNOME Terminal, Konsole, Terminator, XFCE4-Terminal) поддерживают ввод и отображение RTL-текста. Однако Bidirectional (двунаправленный) алгоритм в чистом терминале работает не идеально: арабские буквы могут отображаться в логическом порядке слева направо, без соединения. Для критичных задач используйте текстовый редактор (vim, nano, gedit) вместо cat/echo.

```
user@debian:~$ echo "كتاب — это книга"
كتاب — это книга

user@debian:~$ read -p "Введите арабское: " ar; echo "Вы ввели: $ar"
Введите арабское: سلام
Вы ввели: سلام
```

Если вместо арабского вы видите прямоугольники или знаки вопроса, проблема в шрифте терминала. В GNOME Terminal: «Правка → Параметры → Профили → Текст → Моноширинный шрифт» — выберите шрифт с поддержкой арабского, например

`DejaVu Sans Mono` или `Noto Sans Mono CJK`. Для терминала `xterm` используйте `xterm -fa 'Noto Sans Mono' -en UTF-8`.

Корректный рендеринг RTL через fribidi

Утилита `fribidi` (Free Implementation of the Unicode Bidirectional Algorithm) переупорядочивает арабский текст для отображения в терминалах, не поддерживающих Unicode Bidi. Установите: `sudo apt install fribidi`.

```
user@debian:~$ echo "مرحبا بالعالم" | fribidi
مرحبا بالعالم

user@debian:~$ printf "Арабское: %s\n" "$(echo 'كتاب' | fribidi)"
Арабское: كتاب
```

Для полностью корректного рендеринга с соединением букв и огласовками нужен сложный текстовый движок (Pango, HarfBuzz). В терминале GNOME/Konose это уже сделано, но в скриптах для piped-вывода может потребоваться `fribidi`.

Поддержка локали ar_SA.UTF-8

Чтобы программы корректно работали с арабским (сортировка, регистр), установите арабскую локаль:

```
user@debian:~$ sudo dpkg-reconfigure locales
Появится диалог. Отметьте:
  [x] ar_SA.UTF-8 UTF-8
  [x] ru_RU.UTF-8 UTF-8
  [x] en_US.UTF-8 UTF-8
OK

user@debian:~$ locale -a | grep -E "ar|ru"
ar_SA.utf8
ru_RU.utf8

user@debian:~$ LC_ALL=ar_SA.UTF-8 ls --help | head -3
الاستعمال: ls [خيار]... [ملف]...
```

В скриптах словаря принудительно устанавливайте UTF-8 локаль, чтобы избежать проблем с кодировкой:

УСТАНОВКА ЛОКАЛИ В СКРИПТЕ BASH

```bash
#!/bin/bash
export LC_ALL=ru_RU.UTF-8
export LANG=ru_RU.UTF-8
export LOCALE=UTF-8

# Проверка
if ! locale -a | grep -q "ru_RU.utf8"; then
    echo "Локаль ru_RU.UTF-8 не установлена" >&2
    echo "Выполните: sudo dpkg-reconfigure locales" >&2
    exit 1
fi
```

Арабские буквы и Unicode

Полезно знать основные диапазоны Unicode для арабского письма:

ДИАПАЗОН	БЛОК
U+0600 — U+06FF	Arabic (основные буквы, цифры, огласовки)
U+0750 — U+077F	Arabic Supplement (дополнительные буквы)
U+08A0 — U+08FF	Arabic Extended-A
U+FB50 — U+FDFF	Arabic Presentation Forms-A (лигатуры)
U+FE70 — U+FEFF	Arabic Presentation Forms-B
U+064B — U+0652	Огласовки (tashkil): фатха, kasra, damma и др.

В Bash можно вывести код символа командой `printf`:

```
user@debian:~$ printf '%04x\n' "'ك"
0643

user@debian:~$ printf '\u0643\n'
ك

user@debian:~$ echo -n "كتاب" | xxd
00000000: d983 d8aa d8a7 d8b2                      ........
```

Последняя команда показывает байты UTF-8: арабская буква `ك` кодируется двумя байтами `D9 83`. Знание этого помогает понимать ошибки с кодировкой и корректно

обрабатывать строки в `awk` и `sed`.

Практика

> **ПРАКТИКА К ГЛАВЕ 10**
>
> Установите арабские шрифты (`fonts-noto-core`, `fonts-hosny-amiri`), настройте раскладку `us,ru,ar` с переключением по Alt+Shift, установите локаль `ar_SA.UTF-8`. Проверьте, что в терминале команда `echo "السلام عليكم"` выводит корректный арабский текст. Напишите скрипт, который спрашивает арабское слово, читает его через `read`, и выводит его длину в символах и байтах (используйте `${#var}` и `echo -n "$var" | wc -c`).

ГЛАВА 11 · СУБД

Сравнение СУБД для словаря

Прежде чем углубиться в установку и работу с конкретной СУБД, обсудим, какой выбор подходит для арабо-русского словаря. Существует три популярных реляционных СУБД, доступных в Debian: SQLite, PostgreSQL и MariaDB. Каждая имеет свои сильные стороны и сценарии применения.

Критерии выбора

Для словаря ключевые критерии таковы. Поддержка Unicode — критическая, ведь и арабский, и русский используют не-ASCII символы. Простота установки важна для новичка. Производительность для словаря на 10-100 тысяч слов не критична: любая из трёх СУБД справится мгновенно. Масштабируемость: если вы планируете веб-интерфейс или многопользовательский доступ — нужна клиент-серверная СУБД.

КРИТЕРИЙ	SQLITE	POSTGRESQL	MARIADB
Архитектура	Встраиваемая, без сервера	Клиент-сервер	Клиент-сервер
Установка	`apt install sqlite3`	`apt install postgresql`	`apt install mariadb-server`
Размер пакета	~1 МБ	~120 МБ	~80 МБ
Файл БД	Один файл .db	Каталог в /var/lib/postgresql/	Каталог в /var/lib/mysql/
Unicode/UTF-8	Полная поддержка	Полная + collation по языку	Полная (utf8mb4)
Арабский collation	Через ICU (опционально)	Да, из коробки	Да, через utf8mb4_unicode_ci
Параллельный доступ	Один писатель	Много писателей	Много писателей
Триггеры, views	Да (базовые)	Да (богатые)	Да
Полнотекстовый поиск	FTS5 (хороший)	tsvector (мощный)	FULLTEXT (базовый)
Бэкап	Копия файла	`pg_dump`	`mysqldump`
Лицензия	Public domain	PostgreSQL License (BSD-like)	GPLv2

Когда выбрать SQLite

SQLite — встроенная СУБД: нет сервера, нет сетевого доступа, нет пользователя. Вся база лежит в одном файле, который можно скопировать на флешку, отправить по почте, положить в Git. Это идеальный выбор для личного словаря,desktop-приложений, прототипов, тестов. Если ваш словарь будут использовать только вы с одного компьютера — SQLite лучший выбор. Он быстр, надёжен, не требует обслуживания.

Ограничения: один писатель в любой момент времени (другие читатели могут работать параллельно), нет сетевого доступа, нет аутентификации. Для веб-приложения с тысячами пользователей SQLite не подойдёт.

Когда выбрать PostgreSQL

PostgreSQL — мощная объектно-реляционная СУБД с акцентом на соответствие стандартам и расширяемость. Поддерживает сложные типы данных, JSON, геоданные (через PostGIS), мощный полнотекстовый поиск. Идеальный выбор для production-систем, где важна надёжность, целостность данных и богатые возможности SQL.

Для словаря PostgreSQL даёт две важные вещи. Во-первых, корректный collation для арабского — сортировка по правилам арабского алфавита, а не по кодам Unicode. Во-вторых, мощный полнотекстовый поиск с морфологией (хотя для арабского нужна дополнительная конфигурация). Если вы планируете веб-версию словаря или API — PostgreSQL оправдан.

Когда выбрать MariaDB

MariaDB — форк MySQL, поддерживаемый оригинальными авторами. Популярна в веб-разработке, особенно в связке с PHP (WordPress, Joomla). Простая в установке и освоении, имеет хорошую производительность для типичных веб-запросов. Если вы планируете веб-словарь на PHP или уже знакомы с MySQL — MariaDB будет логичным выбором.

Для арабского важно использовать кодировку `utf8mb4`, а не `utf8` (которая в MySQL/MariaDB поддерживает только BMP до 3 байт). Арабские символы помещаются в BMP, так что `utf8` технически работает, но `utf8mb4` — более правильный и будущий-устойчивый выбор.

Рекомендация для нашего проекта

> **РЕКОМЕНДАЦИЯ**
>
> Для личного арабо-русского словаря используйте **SQLite**. Это самый простой старт: одна команда установки, один файл БД, легко бэкапить. Когда словарь вырастет или понадобится веб-доступ — мигрируйте на PostgreSQL. MariaDB рассматривайте, если уже есть опыт с MySQL или планируется PHP-стек.

В этой книге мы рассмотрим все три СУБД, чтобы вы могли выбрать подходящую. Главы 12-14 показывают установку и базовые операции, глава 15 — единую SQL-схему, работающую во всех трёх. В главах 16-19 Bash-скрипты написаны так, чтобы легко переключаться между СУБД через переменную окружения.

Установка всех трёх СУБД

Если хотите попробовать все три, установите их одной командой. Они не конфликтуют (используют разные порты и файлы):

УСТАНОВКА ВСЕХ ТРЁХ BASH

```bash
sudo apt update
sudo apt install -y sqlite3 postgresql postgresql-client mariadb-server mariadb-client

# Проверка версий
sqlite3 --version
psql --version
mariadb --version
```

Службы PostgreSQL и MariaDB запустятся автоматически. Проверить статус: `sudo systemctl status postgresql mariadb`. SQLite службы не имеет — это просто библиотека и CLI.

Практика

> **ПРАКТИКА К ГЛАВЕ 11**
>
> Установите хотя бы одну СУБД (рекомендуется SQLite для старта). Проверьте версию, запустите CLI, выйдите. Создайте каталог `~/dict-ar-ru/data`, где будет лежать файл БД (для SQLite) или where будут складываться SQL-скрипты (для PostgreSQL/MariaDB). Решите, какую СУБД будете использовать в следующих главах, и зафиксируйте это в файле `scripts/config.sh` в переменной `DB_ENGINE`.

ГЛАВА 12 · SQLITE

SQLite: установка и основы

SQLite — самая простая в установке и использовании СУБД из нашего обзора. Это встраиваемая БД: вместо сервера с сетевым доступом — библиотека, которая работает с файлом напрямую. Идеальный выбор для личного словаря, прототипов, тестов и любых приложений, где не нужен параллельный доступ нескольких процессов на запись.

Установка

```
user@debian:~$ sudo apt install -y sqlite3
user@debian:~$ sqlite3 --version
3.40.1 2022-12-28 14:03:47 df5c73...
```

Дополнительно можно установить графический клиент DB Browser for SQLite: `sudo apt install sqlitebrowser`. Для командной строки есть удобный `litecli` с подсветкой синтаксиса и автодополнением (устанавливается через pip: `pip install litecli`).

Создание базы данных

База данных SQLite — это обычный файл. Чтобы создать новую БД, достаточно передать имя файла в `sqlite3`:

```
user@debian:~/dict-ar-ru$ cd ~/dict-ar-ru
user@debian:~/dict-ar-ru$ sqlite3 data/dict.db
SQLite version 3.40.1 2022-12-28
Enter ".help" for usage hints.
sqlite>

sqlite> .tables
sqlite> .schema
sqlite> .exit
user@debian:~/dict-ar-ru$ ls -lh data/dict.db
-rw-r--r-- 1 user user 12K окт 23 10:30 data/dict.db
```

Файл `dict.db` создан. Внутри пока пусто. Команды, начинающиеся с точки (`.tables`, `.schema`, `.exit`) — это мета-команды оболочки sqlite3, не SQL. Они управляют форматом вывода, импортом/экспортом, настройками.

Полезные dot-команды

КОМАНДА	НАЗНАЧЕНИЕ
`.tables`	Список таблиц
`.schema`	SQL-схема всех таблиц
`.schema words`	Схема конкретной таблицы
`.indexes`	Список индексов
`.databases`	Список подключённых БД
`.mode csv`	Режим вывода CSV
`.mode column`	Табличный вывод
`.headers on`	Показывать заголовки колонок
`.read file.sql`	Выполнить SQL из файла
`.import file.csv table`	Импорт CSV в таблицу
`.dump`	Дамп всей БД в SQL
`.exit` или `.quit`	Выйти

Создание таблицы

Создадим таблицу `words` для нашего словаря. SQL-команды можно вводить прямо в интерактивной оболочке, заканчивая точкой с запятой.

СОЗДАНИЕ ТАБЛИЦЫ SQL

```
sqlite> CREATE TABLE words (
   ...>     id INTEGER PRIMARY KEY AUTOINCREMENT,
   ...>     arabic TEXT NOT NULL,
   ...>     russian TEXT NOT NULL,
   ...>     transcription TEXT,
   ...>     part_of_speech TEXT,
   ...>     created_at TEXT DEFAULT (datetime('now'))
   ...> );
sqlite> CREATE INDEX idx_arabic ON words(arabic);
sqlite> CREATE INDEX idx_russian ON words(russian);
sqlite> .tables
words
sqlite> .schema words
CREATE TABLE words (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    arabic TEXT NOT NULL,
    russian TEXT NOT NULL,
    transcription TEXT,
    part_of_speech TEXT,
    created_at TEXT DEFAULT (datetime('now'))
);
CREATE INDEX idx_arabic ON words(arabic);
CREATE INDEX idx_russian ON words(russian);
```

Тип `TEXT` в SQLite универсальный — хранит строки в UTF-8, включая арабские и русские символы, без дополнительных настроек.
`INTEGER PRIMARY KEY AUTOINCREMENT` — автоинкрементный числовой идентификатор.
`DEFAULT (datetime('now'))` — автоматически подставляет текущее время.

Основные SQL-операции

CRUD В SQLITE — SQL

```sql
-- Вставка
INSERT INTO words (arabic, russian, transcription, part_of_speech)
VALUES ('كتاب', 'книга', 'kitab', 'существительное');

INSERT INTO words (arabic, russian, transcription) VALUES
    ('باب', 'дверь', 'bab'),
    ('ماء', 'вода', 'maa'),
    ('سلام', 'мир', 'salaam');

-- Выборка
SELECT * FROM words;
SELECT arabic, russian FROM words WHERE part_of_speech = 'существительное';
SELECT * FROM words ORDER BY arabic LIMIT 5;

-- Поиск
SELECT * FROM words WHERE arabic LIKE '%ك%';
SELECT * FROM words WHERE russian LIKE 'к%';

-- Обновление
UPDATE words SET transcription = 'kitaab' WHERE arabic = 'كتاب';

-- Удаление
DELETE FROM words WHERE id = 3;
```

Выполнение SQL из командной строки

Часто удобнее запускать SQL одной командой, без входа в интерактивный режим. Есть два способа.

CLI РЕЖИМ BASH

```bash
# Один запрос как аргумент
sqlite3 data/dict.db "SELECT * FROM words LIMIT 5;"

# SQL из here-doc
sqlite3 data/dict.db <<SQL
.headers on
.mode column
SELECT arabic, russian, transcription FROM words ORDER BY arabic;
SQL

# SQL из файла
sqlite3 data/dict.db < scripts/schema.sql

-- Файл scripts/schema.sql
DROP TABLE IF EXISTS words;
CREATE TABLE words (...);
CREATE INDEX ...;
```

Unicode и арабский в SQLite

SQLite хранит строки в UTF-8 без настройки кодировки — это упрощает жизнь. Однако сортировка по умолчанию идёт по бинарным кодам символов, а не по правилам языка. Для корректной сортировки арабских и русских слов используйте `COLLATE` с ICU-сопоставлением (если SQLite собран с ICU, что в Debian по умолчанию).

СОРТИРОВКА ПО ЛОКАЛИ SQL

```sql
-- По умолчанию: бинарная сортировка
SELECT arabic FROM words ORDER BY arabic;

-- С ICU collation (если доступен)
SELECT arabic FROM words ORDER BY arabic COLLATE NOCASE;

-- Для русского — тоже NOCASE подойдёт для регистра
SELECT russian FROM words ORDER BY russian COLLATE NOCASE;
```

Если ICU недоступен, сортировка по правилам языка делается на стороне приложения (например, через `sort` с установленной локалью в Bash).

Практика

> **ПРАКТИКА К ГЛАВЕ 12**
>
> Установите SQLite, создайте файл БД `data/dict.db`, примените схему из главы 15 (или напишите свою). Добавьте 5-10 слов через SQL `INSERT`. Сделайте выборки: все слова, слова на букву «к», слова с определённой частью речи. Сохраните дамп БД в файл `backup/dict_dump.sql` командой `sqlite3 data/dict.db .dump > backup/dict_dump.sql`.

ГЛАВА 13 · POSTGRESQL

PostgreSQL: установка и основы

PostgreSQL — мощная объектно-реляционная СУБД с акцентом на соответствие стандартам SQL, расширяемость и надёжность. В отличие от SQLite, PostgreSQL работает как сервер: имеет процесс-демон, слушает сетевой порт (5432 по умолчанию), поддерживает аутентификацию и многопользовательский доступ. Для production-систем и веб-приложений PostgreSQL — один из лучших выборов.

Установка

```
user@debian:~$ sudo apt install -y postgresql postgresql-client
user@debian:~$ psql --version
psql (PostgreSQL) 15.5 (Debian 15.5-0+deb12u1)

user@debian:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (...; enabled; preset: enabled)
     Active: active (exited) since ...
```

После установки PostgreSQL создаёт системного пользователя `postgres`, который является суперпользователем БД. В Debian аутентификация по умолчанию — peer: локальный пользователь `postgres` может войти в БД без пароля, другие пользователи должны соответствовать системным именам.

Создание пользователя и базы

Чтобы работать с PostgreSQL от своего обычного пользователя, создайте для него роль и базу:

```
user@debian:~$ sudo -u postgres psql
psql (15.5)
Введите "help", чтобы получить справку.

postgres=#

postgres=# CREATE USER user WITH PASSWORD 'secret';
CREATE ROLE
postgres=# CREATE DATABASE dict_ar_ru OWNER user ENCODING 'UTF8' LC_COLLATE
'ru_RU.UTF-8' LC_CTYPE 'ru_RU.UTF-8' TEMPLATE template0;
CREATE DATABASE
postgres=# \q
user@debian:~$
```

Здесь мы создали пользователя `user` (замените на ваш логин) с паролем `secret` (придумайте свой), и базу `dict_ar_ru` с UTF-8 и русской локалью. Если локали `ru_RU.UTF-8` нет, сначала сгенерируйте её через `sudo dpkg-reconfigure locales`.

Подключение и psql

Клиент `psql` — основной инструмент работы с PostgreSQL. Полезные ключи: `-h` (хост, по умолчанию локальный Unix-сокет), `-U` (пользователь), `-d` (база), `-c` (выполнить команду), `-f` (выполнить файл).

```
user@debian:~$ psql -d dict_ar_ru
psql (15.5)
Введите "help", чтобы получить справку.

dict_ar_ru=>

dict_ar_ru=> \dt                  # список таблиц
dict_ar_ru=> \d words              # структура таблицы
dict_ar_ru=> \l                    # список баз
dict_ar_ru=> \du                   # список пользователей
dict_ar_ru=> \q                    # выход
```

Создание таблицы с правильным collation

SCHEMA.SQL ДЛЯ POSTGRESQL SQL

```
CREATE TABLE words (
    id SERIAL PRIMARY KEY,
    arabic TEXT NOT NULL,
    russian TEXT NOT NULL,
    transcription TEXT,
    part_of_speech TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Индексы для быстрого поиска
CREATE INDEX idx_arabic ON words(arabic);
CREATE INDEX idx_russian ON words(russian);

-- Уникальность пары arabic+russian (чтобы не было дублей)
CREATE UNIQUE INDEX idx_unique_pair ON words(arabic, russian);
```

Примените схему:

```
user@debian:~/dict-ar-ru$ psql -d dict_ar_ru -f data/schema.sql
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE UNIQUE INDEX

user@debian:~/dict-ar-ru$ psql -d dict_ar_ru -c "INSERT INTO words (arabic,
russian, transcription) VALUES ('كتاب', 'книга', 'kitab');"
INSERT 0 1

user@debian:~/dict-ar-ru$ psql -d dict_ar_ru -c "SELECT * FROM words;"
 id | arabic | russian | transcription | part_of_speech |     created_at
----+--------+---------+---------------+----------------+---------------------
  1 | كتاب   | книга   | kitab         |                | 2026-01-23 10:35:00
(1 строка)
```

PostgreSQL корректно отображает арабские символы и хранит их в UTF-8. Сортировка по умолчанию идёт по правилам локали, указанной при создании базы — то есть для арабского и русского будет корректный алфавитный порядок.

PSQL из Bash-скриптов

PSQL В BASH — BASH

```bash
#!/bin/bash
# pg_dict.sh — работа со словарём в PostgreSQL

PG_DB="dict_ar_ru"
PG_USER="$USER"

pg_add() {
    local ar="$1" ru="$2" tr="$3"
    psql -U "$PG_USER" -d "$PG_DB" -c \
        "INSERT INTO words (arabic, russian, transcription) VALUES ('$ar', '$ru',
'$tr');"
}

pg_search() {
    psql -U "$PG_USER" -d "$PG_DB" -c \
        "SELECT * FROM words WHERE arabic LIKE '%$1%' OR russian LIKE '%$1%';"
}

pg_list() {
    psql -U "$PG_USER" -d "$PG_DB" -c "SELECT * FROM words ORDER BY arabic;"
}

case "$1" in
    add)    pg_add "$2" "$3" "$4" ;;
    search) pg_search "$2" ;;
    list)   pg_list ;;
esac
```

Для больших SQL-запросов удобнее передавать их через stdin или файл, а не через `-c`:

SQL ЧЕРЕЗ STDIN — BASH

```bash
psql -U "$PG_USER" -d "$PG_DB" <<SQL
.mode column
.headers on
SELECT arabic, russian, transcription
FROM words
WHERE part_of_speech = 'существительное'
ORDER BY arabic;
SQL
```

> **ПРЕДУПРЕЖДЕНИЕ О ПАРОЛЯХ**
>
> По умолчанию `psql` спросит пароль интерактивно. В скриптах это неудобно. Используйте файл `~/.pgpass` с правами 600: `hostname:port:database:username:password`. Или переменную `PGPASSWORD=secret psql ...` (но пароль будет виден в `/proc/PID/environ`).

Практика

> **ПРАКТИКА К ГЛАВЕ 13**
>
> Установите PostgreSQL, создайте пользователя с вашим именем и базу `dict_ar_ru`. Примените схему из главы 15. Добавьте 5-10 слов. Сделайте выборки с сортировкой по арабскому и русскому полям. Создайте файл `~/.pgpass` для беспарольного доступа. Напишите скрипт `pg_dict.sh` с подкомандами add/search/list.

ГЛАВА 14 · MARIADB

MariaDB: установка и основы

MariaDB — форк MySQL, развиваемый оригинальными авторами после приобретения MySQL компанией Oracle. Полностью совместима с MySQL на уровне протокола и SQL-синтаксиса, но имеет ряд улучшений в производительности и функциональности. В Debian 12 MariaDB заменяет MySQL в репозиториях — пакет `mysql-server` фактически устанавливает MariaDB.

Установка

```
user@debian:~$ sudo apt install -y mariadb-server mariadb-client
user@debian:~$ mariadb --version
mariadb  Ver 15.1 Distrib 10.11.4-MariaDB, for debian-linux-gnu (x86_64)

user@debian:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.11.4-MariaDB
     Active: active (running) since ...

user@debian:~$ sudo mysql_secure_installation
 Enter current password for root (enter for none):
 Set root password? [Y/n] y
 Remove anonymous users? [Y/n] y
 Disallow root login remotely? [Y/n] y
 Remove test database and access to it? [Y/n] y
 Reload privilege tables now? [Y/n] y
 All done!
```

`mysql_secure_installation` — обязательный скрипт после установки. Он задаёт пароль root, удаляет анонимного пользователя и тестовую базу, запрещает удалённый root-доступ. Пройдите все шаги.

Создание пользователя и базы

```
user@debian:~$ sudo mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
...

MariaDB [(none)]> CREATE DATABASE dict_ar_ru CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> CREATE USER 'user'@'localhost' IDENTIFIED BY 'secret';
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON dict_ar_ru.* TO 'user'@'localhost';
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> exit
```

Важно: используйте кодировку `utf8mb4`, а не `utf8`. В MySQL/MariaDB `utf8` — это устаревший вариант, поддерживающий только 3 байта на символ (без эмодзи и некоторых редких символов). `utf8mb4` — полноценный UTF-8. Для арабского и русского `utf8` технически работает, но `utf8mb4` — современный стандарт.

Клиент mariadb

```
user@debian:~$ mariadb -u user -p dict_ar_ru
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
...

MariaDB [dict_ar_ru]> SHOW TABLES;
MariaDB [dict_ar_ru]> DESCRIBE words;
MariaDB [dict_ar_ru]> SHOW CREATE TABLE words;
MariaDB [dict_ar_ru]> exit
```

Создание таблицы

SCHEMA.SQL ДЛЯ MARIADB SQL

```
CREATE TABLE words (
    id INT AUTO_INCREMENT PRIMARY KEY,
    arabic VARCHAR(255) NOT NULL,
    russian VARCHAR(255) NOT NULL,
    transcription VARCHAR(255),
    part_of_speech VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_arabic (arabic),
    INDEX idx_russian (russian),
    UNIQUE KEY unique_pair (arabic, russian)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Примените схему:

```
user@debian:~/dict-ar-ru$ mariadb -u user -p dict_ar_ru < data/schema.sql
Enter password:

user@debian:~/dict-ar-ru$ mariadb -u user -p dict_ar_ru -e "INSERT INTO words
(arabic, russian, transcription) VALUES ('كتاب', 'книга', 'kitab');"
Enter password:

user@debian:~/dict-ar-ru$ mariadb -u user -p dict_ar_ru -e "SELECT * FROM words;"
+----+--------+---------+---------------+---------------+---------------------+
| id | arabic | russian | transcription | part_of_speech | created_at          |
+----+--------+---------+---------------+---------------+---------------------+
|  1 | كتاب   | книга   | kitab         | NULL           | 2026-01-23 10:40:00 |
+----+--------+---------+---------------+---------------+---------------------+
```

Флаг `-e` выполняет SQL-команду и сразу выходит. Для пароля можно использовать `--password=secret` или файл `~/.my.cnf` с секцией `[client]`.

Файл ~/.my.cnf для беспарольного доступа

~/.MY.CNF CONFIG

```
[client]
user = user
password = secret
host = localhost

[mysqldump]
user = user
password = secret
```

Установите права 600: `chmod 600 ~/.my.cnf`. Теперь `mariadb dict_ar_ru` будет подключаться без пароля.

Bash-скрипты для MariaDB

MY_DICT.SH BASH

```
#!/bin/bash
# my_dict.sh — работа со словарём в MariaDB

MY_DB="dict_ar_ru"

my_add() {
    mariadb "$MY_DB" -e \
        "INSERT INTO words (arabic, russian, transcription) VALUES ('$1', '$2',
'$3');"
}

my_search() {
    mariadb "$MY_DB" -e \
        "SELECT * FROM words WHERE arabic LIKE '%$1%' OR russian LIKE '%$1%';"
}

my_list() {
    mariadb "$MY_DB" -e "SELECT * FROM words ORDER BY arabic;"
}

case "$1" in
    add)    my_add "$2" "$3" "$4" ;;
    search) my_search "$2" ;;
    list)   my_list ;;
esac
```

Практика

> **ПРАКТИКА К ГЛАВЕ 14**
>
> Установите MariaDB, выполните `mysql_secure_installation`, создайте базу `dict_ar_ru` с кодировкой `utf8mb4` и пользователя с паролем. Создайте `~/.my.cnf` для беспарольного доступа. Примените схему из главы 15. Добавьте 5-10 слов, сделайте выборки. Напишите скрипт `my_dict.sh`.

ГЛАВА 15 · СХЕМА

SQL-схема словаря

Схема базы данных — это структура таблиц, индексов и связей между ними. Хорошая схема — основа производительного и поддерживаемого приложения. Для арабо-русского словаря мы спроектируем схему, которая поддерживает не только пары слов, но и категории, примеры употребления, транскрипцию и историю изменений.

Анализ предметной области

Что мы хотим хранить? Каждая запись — это арабское слово, его русский перевод, транскрипция латиницей (для тех, кто не читает по-арабски), часть речи (существительное, глагол, прилагательное и т. д.). Хочется хранить примеры употребления — фразы на арабском с переводом, чтобы показать слово в контексте. Полезна категоризация: еда, путешествия, бизнес, религия. И timestamp — когда добавлено, когда изменено.

Связи между сущностями: одно слово может иметь несколько примеров (one-to-many). Одно слово может принадлежать нескольким категориям, и в одной категории много слов (many-to-many). Это требует таблицы-связки `word_categories`.

Диаграмма схемы



Универсальная схема

Ниже SQL-схема, работающая во всех трёх СУБД с минимальными изменениями. Основные отличия: тип автоинкрементного PK (`SERIAL` в PostgreSQL, `AUTO_INCREMENT` в MariaDB, `AUTOINCREMENT` в SQLite) и функция текущего времени (`NOW()`, `CURRENT_TIMESTAMP`, `datetime('now')`).

SCHEMA.SQL (SQLITE-ВЕРСИЯ) SQL

```sql
-- Главная таблица: слова
CREATE TABLE IF NOT EXISTS words (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    arabic TEXT NOT NULL,
    russian TEXT NOT NULL,
    transcription TEXT,
    part_of_speech TEXT CHECK (
        part_of_speech IN (
            'существительное', 'глагол', 'прилагательное',
            'наречие', 'местоимение', 'предлог',
            'союз', 'частица', 'междометие'
        )
    ),
    notes TEXT,
    created_at TEXT DEFAULT (datetime('now')),
    updated_at TEXT DEFAULT (datetime('now'))
);

-- Категории (еда, путешествия, и т.д.)
CREATE TABLE IF NOT EXISTS categories (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE,
    description TEXT
);

-- Примеры употребления
CREATE TABLE IF NOT EXISTS examples (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    word_id INTEGER NOT NULL,
    arabic_text TEXT NOT NULL,
    russian_text TEXT NOT NULL,
    FOREIGN KEY (word_id) REFERENCES words(id) ON DELETE CASCADE
);

-- Связь many-to-many: слово ↔ категория
CREATE TABLE IF NOT EXISTS word_categories (
    word_id INTEGER NOT NULL,
    category_id INTEGER NOT NULL,
    PRIMARY KEY (word_id, category_id),
    FOREIGN KEY (word_id) REFERENCES words(id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
);

-- Индексы для быстрого поиска
CREATE INDEX IF NOT EXISTS idx_words_arabic ON words(arabic);
CREATE INDEX IF NOT EXISTS idx_words_russian ON words(russian);
CREATE INDEX IF NOT EXISTS idx_words_pos ON words(part_of_speech);
CREATE INDEX IF NOT EXISTS idx_examples_word ON examples(word_id);
```

Версия для PostgreSQL

SCHEMA_PG.SQL — ОТЛИЧИЯ SQL

```sql
-- Заменить типы PK и времени
id SERIAL PRIMARY KEY,                          -- вместо INTEGER ... AUTOINCREMENT
created_at TIMESTAMP DEFAULT NOW(),             -- вместо datetime('now')
updated_at TIMESTAMP DEFAULT NOW()

-- Триггер в PostgreSQL пишется иначе
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_words_updated
BEFORE UPDATE ON words
FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

Версия для MariaDB

SCHEMA_MY.SQL — ОТЛИЧИЯ SQL

```sql
-- Заменить типы
id INT AUTO_INCREMENT PRIMARY KEY,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

-- В MariaDB updated_at обновляется автоматически через ON UPDATE
-- Триггер не нужен!

-- ENGINE и CHARSET в конце CREATE TABLE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- CHECK в MariaDB 10.2+ работает, но можно опустить
```

Применение схемы

ПРИМЕНЕНИЕ В КАЖДОЙ СУБД — BASH

```bash
# SQLite
sqlite3 data/dict.db < data/schema_sqlite.sql

# PostgreSQL
psql -U "$USER" -d dict_ar_ru -f data/schema_pg.sql

# MariaDB
mariadb dict_ar_ru < data/schema_my.sql
```

Примеры SQL-запросов

СЛОЖНЫЕ ЗАПРОСЫ SQL

```sql
-- Слова с примерами
SELECT w.arabic, w.russian, e.arabic_text, e.russian_text
FROM words w
LEFT JOIN examples e ON w.id = e.word_id
WHERE w.arabic LIKE 'ك%';

--  Слова по категории
SELECT w.arabic, w.russian
FROM words w
JOIN word_categories wc ON w.id = wc.word_id
JOIN categories c ON wc.category_id = c.id
WHERE c.name = 'еда';

-- Все категории слова
SELECT c.name
FROM categories c
JOIN word_categories wc ON c.id = wc.category_id
WHERE wc.word_id = 5;

-- Статистика по частям речи
SELECT part_of_speech, COUNT(*) as count
FROM words
GROUP BY part_of_speech
ORDER BY count DESC;

-- Поиск по подстроке в арабском или русском
SELECT * FROM words
WHERE arabic LIKE '%سلام%' OR russian LIKE '%мир%';

-- Добавить категорию к слову
INSERT INTO word_categories (word_id, category_id)
VALUES (1, (SELECT id FROM categories WHERE name = 'еда'));
```

Практика

> **ПРАКТИКА К ГЛАВЕ 15**
>
> Создайте файл `data/schema.sql` с полной схемой для вашей СУБД (SQLite, PostgreSQL или MariaDB). Примените схему. Добавьте 2-3 категории (еда, путешествия, базовые слова). Добавьте 5 слов и свяжите их с категориями. Добавьте 2-3 примера употребления. Выполните запросы: все слова в категории «еда», все категории конкретного слова, статистика по частям речи.

ГЛАВА 16 · BASH-СКРИПТЫ

Bash-скрипты для работы со словарём

Теперь объединим всё изученное: Bash, SQLite/PostgreSQL/MariaDB, функции, обработку ошибок — в один проект. Цель: набор скриптов, которые позволяют добавлять слова, искать, перечислять, удалять и обновлять записи в арабо-русском словаре. Скрипты будут поддерживать все три СУБД через переменную окружения `DB_ENGINE`.

Структура проекта

```
dict-ar-ru/
├── scripts/
│   ├── dict.sh              # главный скрипт
│   ├── config.sh            # конфигурация (пути, СУБД)
│   ├── lib/
│   │   ├── db_sqlite.sh     # функции для SQLite
│   │   ├── db_pg.sh         # функции для PostgreSQL
│   │   └── db_my.sh         # функции для MariaDB
│   ├── add_word.sh          # интерактивное добавление
│   ├── search.sh            # поиск
│   └── list.sh              # листинг
├── data/
│   ├── dict.db              # БД SQLite (или симлинк)
│   ├── schema_sqlite.sql
│   ├── schema_pg.sql
│   └── schema_my.sql
├── backup/                  # резервные копии
├── logs/                    # логи
└── docs/
    └── README.md
```

Конфигурация

SCRIPTS/CONFIG.SH BASH

```bash
#!/bin/bash
# config.sh — общая конфигурация проекта

# Локаль — критично для арабского и русского
export LC_ALL=ru_RU.UTF-8
export LANG=ru_RU.UTF-8

# Корень проекта: каталог, где лежит scripts/
export PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

# Тип БД: sqlite, postgresql, mariadb
export DB_ENGINE="${DB_ENGINE:-sqlite}"

# Параметры SQLite
export SQLITE_DB="$PROJECT_ROOT/data/dict.db"

# Параметры PostgreSQL
export PG_DB="dict_ar_ru"
export PG_USER="$USER"

# Параметры MariaDB
export MY_DB="dict_ar_ru"

# Логи
export LOG_DIR="$PROJECT_ROOT/logs"
mkdir -p "$LOG_DIR"

# Цвета для вывода (если терминал поддерживает)
if [[ -t 1 ]]; then
    export CLR_GREEN=$'\033[32m'
    export CLR_RED=$'\033[31m'
    export CLR_YELLOW=$'\033[33m'
    export CLR_BLUE=$'\033[34m'
    export CLR_RESET=$'\033[0m'
else
    export CLR_GREEN= CLR_RED= CLR_YELLOW= CLR_BLUE= CLR_RESET=
fi

# Логирование
log() {
    local level="$1"; shift
    local msg="$*"
    local timestamp=$(date +'%Y-%m-%d %H:%M:%S')
    echo "[$timestamp] [$level] $msg" >> "$LOG_DIR/dict.log"
    if [[ "$level" == "ERROR" ]]; then
        echo -e "${CLR_RED}[$level] $msg${CLR_RESET}" >&2
    elif [[ "$level" == "WARN" ]]; then
        echo -e "${CLR_YELLOW}[$level] $msg${CLR_RESET}" >&2
```

Абстракция БД: db.sh

Создадим слой абстракции, который скрывает различия между СУБД. Каждая функция проверяет `$DB_ENGINE` и вызывает соответствующую реализацию.

SCRIPTS/LIB/DB.SH — СЛОЙ АБСТРАКЦИИ — BASH

```bash
#!/bin/bash
# lib/db.sh — абстракция над СУБД

db_query() {
    # $1 — SQL, опционально $2 — формат вывода
    local sql="$1"
    case "$DB_ENGINE" in
        sqlite)
            sqlite3 "$SQLITE_DB" "$sql"
            ;;
        postgresql)
            psql -t -U "$PG_USER" -d "$PG_DB" -c "$sql"
            ;;
        mariadb)
            mariadb "$MY_DB" -N -e "$sql"
            ;;
        *)
            log ERROR "Неизвестная СУБД: $DB_ENGINE"
            return 1
            ;;
    esac
}

db_query_pretty() {
    # Вывод в виде красивой таблицы
    local sql="$1"
    case "$DB_ENGINE" in
        sqlite)     sqlite3 -header -column "$SQLITE_DB" "$sql" ;;
        postgresql) psql -U "$PG_USER" -d "$PG_DB" -c "$sql" ;;
        mariadb)    mariadb "$MY_DB" -t -e "$sql" ;;
    esac
}

db_escape() {
    # Экранирование одинарных кавычек для SQL
    local s="$1"
    echo "${s//\'/\'\'}"
}
```

Функции для словаря

SCRIPTS/LIB/DICT.SH — ОПЕРАЦИИ СЛОВАРЯ BASH

```bash
#!/bin/bash
# lib/dict.sh — high-level операции

dict_add() {
    local ar="$(db_escape "$1")"
    local ru="$(db_escape "$2")"
    local tr="$(db_escape "$3")"
    local pos="$(db_escape "$4")"

    [[ -z "$ar" || -z "$ru" ]] && {
        log ERROR "Требуются арабское и русское слова"
        return 1
    }

    db_query "INSERT INTO words (arabic, russian, transcription, part_of_speech)
              VALUES ('$ar', '$ru', '$tr', '$pos');" \
        && log INFO "Добавлено: $ar → $ru"
}

dict_search() {
    local q="$(db_escape "$1")"
    db_query_pretty "SELECT id, arabic, russian, transcription, part_of_speech
        FROM words
        WHERE arabic LIKE '%$q%' OR russian LIKE '%$q%' OR transcription LIKE
'%$q%'
        ORDER BY arabic;"
}

dict_list() {
    db_query_pretty "SELECT id, arabic, russian, transcription
        FROM words ORDER BY arabic;"
}

dict_count() {
    db_query "SELECT COUNT(*) FROM words;"
}

dict_delete() {
    local id="$1"
    db_query "DELETE FROM words WHERE id = $id;" \
        && log INFO "Удалена запись #$id"
}

dict_update() {
    local id="$1"
    local field="$(db_escape "$2")"
    local value="$(db_escape "$3")"
```

Главный скрипт dict.sh

SCRIPTS/DICT.SH BASH

```bash
#!/bin/bash
# dict.sh — главный скрипт словаря
set -euo pipefail

# Подключаем конфиг и библиотеки
source "$(dirname "${BASH_SOURCE[0]}")/config.sh"
source "$PROJECT_ROOT/scripts/lib/db.sh"
source "$PROJECT_ROOT/scripts/lib/dict.sh"

usage() {
    cat <<EOF
Использование: dict.sh <команда> [аргументы]

Команды:
  add <ar> <ru> [tr] [pos]   Добавить слово
  search <запрос>             Поиск по подстроке
  list                        Показать все слова
  count                       Количество записей
  delete <id>                 Удалить по ID
  update <id> <field> <val>   Обновить поле
  help                        Эта справка

Переменные окружения:
  DB_ENGINE   sqlite (по умолчанию) | postgresql | mariadb

Примеры:
  dict.sh add كتاب книга kitab существительное
  dict.sh search книга
  dict.sh list
  DB_ENGINE=postgresql dict.sh count
EOF
}

# Если нет аргументов — показать справку
[[ $# -eq 0 ]] && { usage; exit 0; }

case "$1" in
    add)
        shift
        dict_add "$@"
        ;;
    search)
        dict_search "$2"
        ;;
    list)
        dict_list
        ;;
    count)
```

Использование

```
user@debian:~/dict-ar-ru$ chmod +x scripts/dict.sh
user@debian:~/dict-ar-ru$ ./scripts/dict.sh add كتاب книга kitab существительное
user@debian:~/dict-ar-ru$ ./scripts/dict.sh add باب باب bab существительное
user@debian:~/dict-ar-ru$ ./scripts/dict.sh add سلام мир salaam существительное
user@debian:~/dict-ar-ru$ ./scripts/dict.sh count
Записей в словаре: 3
user@debian:~/dict-ar-ru$ ./scripts/dict.sh search мир
id  arabic  russian  transcription  part_of_speech
--- ------- -------- -------------- ---------------
3    سلام   мир      salaam         существительное
user@debian:~/dict-ar-ru$ DB_ENGINE=postgresql ./scripts/dict.sh list
(вывод из PostgreSQL)
```

Практика

> **ПРАКТИКА К ГЛАВЕ 16**
>
> Соберите проект по структуре из начала главы. Создайте `config.sh`, `lib/db.sh`, `lib/dict.sh`, `dict.sh`. Примените схему из главы 15. Добавьте 5-10 слов. Проверьте все подкоманды: add, search, list, count, update, delete. Переключите `DB_ENGINE` и проверьте, что всё работает с другой СУБД.

ГЛАВА 17 · ИМПОРТ CSV

Импорт данных из CSV

Вводить слова по одному через форму — медленно. Обычно словарь импортируют из готового CSV-файла: либо собственного, либо скачанного из открытых источников. В этой главе разберём, как подготовить CSV и импортировать его в каждую из трёх СУБД.

Формат CSV

CSV (Comma-Separated Values) — простейший формат табличных данных. Каждая строка — запись, поля разделены запятыми, первая строка обычно — заголовок. Для нашего словаря:

DATA/WORDS.CSV — CSV

```
arabic,russian,transcription,part_of_speech
كتاب,книга,kitab,существительное
باب,дверь,bab,существительное
ماء,вода,maa,существительное
سلام,мир,salaam,существительное
يأكل,есть,yakul,глагол
كبير,большой,kabir,прилагательное
جميل,красивый,jamil,прилагательное
أنا,я,ana,местоимение
أين,где,ayn,вопросительное слово
شكرا,спасибо,shukran,междометие
```

Если в данных есть запятые внутри значений (например, «спасибо, благодарю»), это поле заключается в двойные кавычки: `"спасибо, благодарю"`. Если в значении есть кавычки — они удваиваются: `"он сказал ""привет"""`.

Файл должен быть в UTF-8 без BOM. Проверить:

```
user@debian:~/dict-ar-ru$ file data/words.csv
data/words.csv: Unicode text, UTF-8 text

user@debian:~/dict-ar-ru$ head -3 data/words.csv | hexdump -C | head -5
00000000  61 72 61 62 69 63 2c 72  75 73 73 69 61 6e 2c 74  |arabic,russian,t|
00000010  72 61 6e 73 63 72 69 70  74 69 6f 6e 2c 70 61 72  |ranscription,par|
... (нет EF BB BF в начале — значит без BOM)
```

Импорт в SQLite

SQLite имеет встроенную команду `.import` для CSV. Важно настроить режим CSV, чтобы корректно обрабатывались кавычки.

ИМПОРТ В SQLITE — BASH

```bash
# Способ 1: через интерактивный режим
sqlite3 data/dict.db
sqlite> .mode csv
sqlite> .import --skip 1 data/words.csv words

# Способ 2: через here-doc
sqlite3 data/dict.db <<SQL
.mode csv
.import --skip 1 data/words.csv words
SQL

# Способ 3: Bash-скрипт с построчной обработкой
# (если нужен контроль или трансформация данных)
while IFS=, read -r ar ru tr pos; do
    [[ "$ar" == "arabic" ]] && continue   # пропустить заголовок
    sqlite3 data/dict.db \
        "INSERT INTO words (arabic, russian, transcription, part_of_speech) VALUES
('$ar', '$ru', '$tr', '$pos');"
done < data/words.csv
```

Флаг `--skip 1` пропускает заголовок. В старых версиях SQLite (до 3.23) этого флага нет — заголовок приходится пропускать вручную через `tail -n +2`.

Импорт в PostgreSQL

PostgreSQL предлагает команду `COPY` — одну из самых быстрых для массовой загрузки. Есть серверная `COPY` (файл должен быть доступен серверу) и клиентская `\copy` (файл читается клиентом, доступен локально).

ИМПОРТ В POSTGRESQL BASH

```bash
# Способ 1: \copy из psql (рекомендуемый для локальных файлов)
psql -U "$USER" -d dict_ar_ru -c \
    "\copy words(arabic, russian, transcription, part_of_speech) FROM
'data/words.csv' WITH (FORMAT csv, HEADER true);"

# Способ 2: COPY FROM STDIN через here-doc
psql -U "$USER" -d dict_ar_ru <<SQL
\copy words(arabic, russian, transcription, part_of_speech) FROM 'data/words.csv'
WITH (FORMAT csv, HEADER true)
SQL

# Способ 3: серверный COPY (требует прав суперпользователя)
psql -U postgres -d dict_ar_ru -c \
    "COPY words(arabic, russian, transcription, part_of_speech) FROM
'/abs/path/words.csv' WITH (FORMAT csv, HEADER true);"
```

Параметр `FORMAT csv` включает правильную обработку кавычек. `HEADER true` пропускает первую строку. Для путей обязательно указывать абсолютный путь при серверном COPY.

Импорт в MariaDB

MariaDB использует `LOAD DATA INFILE`. По умолчанию файл должен лежать в каталоге, указанном в переменной `secure_file_priv` (проверить: `SHOW VARIABLES LIKE 'secure_file_priv';`). Если значение пустое — можно грузить откуда угодно, если NULL — загрузка запрещена.

ИМПОРТ В MARIADB BASH

```
# Способ 1: LOAD DATA LOCAL INFILE (файл на клиенте, не требует прав сервера)
mariadb --local-infile=1 dict_ar_ru -e "
LOAD DATA LOCAL INFILE 'data/words.csv'
INTO TABLE words
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '\"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(arabic, russian, transcription, part_of_speech);
"

# Способ 2: если secure_file_priv пустой
mariadb dict_ar_ru -e "
LOAD DATA INFILE '/abs/path/words.csv'
INTO TABLE words
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '\"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(arabic, russian, transcription, part_of_speech);
"
```

Параметр `CHARACTER SET utf8mb4` критичен — без него MariaDB может неверно интерпретировать байты. `OPTIONALLY ENCLOSED BY '\"'` указывает, что поля могут быть в кавычках. `IGNORE 1 ROWS` пропускает заголовок.

Универсальный скрипт импорта

SCRIPTS/IMPORT_CSV.SH BASH

```bash
#!/bin/bash
# import_csv.sh — универсальный импорт CSV
set -euo pipefail

source "$(dirname "$0")/config.sh"
source "$PROJECT_ROOT/scripts/lib/db.sh"

CSV_FILE="${1:-$PROJECT_ROOT/data/words.csv}"

[[ ! -f "$CSV_FILE" ]] && {
    log ERROR "Файл не найден: $CSV_FILE"
    exit 1
}

log INFO "Импорт $CSV_FILE в $DB_ENGINE..."

case "$DB_ENGINE" in
    sqlite)
        sqlite3 "$SQLITE_DB" <<SQL
.mode csv
.import --skip 1 "$CSV_FILE" words
SQL
        ;;
    postgresql)
        psql -U "$PG_USER" -d "$PG_DB" -c \
            "\\copy words(arabic, russian, transcription, part_of_speech) FROM
'$CSV_FILE' WITH (FORMAT csv, HEADER true);"
        ;;
    mariadb)
        mariadb --local-infile=1 "$MY_DB" -e "
LOAD DATA LOCAL INFILE '$CSV_FILE'
INTO TABLE words
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '\"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(arabic, russian, transcription, part_of_speech);
"
        ;;
    *)
        log ERROR "Неизвестная СУБД: $DB_ENGINE"
        exit 1
        ;;
esac

log INFO "Импорт завершён. Записей: $(db_query 'SELECT COUNT(*) FROM words;')"
```

Экспорт в CSV

Обратная операция — выгрузка словаря в CSV для бэкапа или обмена.

ЭКСПОРТ BASH

```bash
# SQLite
sqlite3 -header -csv data/dict.db \
    "SELECT arabic, russian, transcription, part_of_speech FROM words;" \
    > data/export.csv

# PostgreSQL
psql -U "$USER" -d dict_ar_ru -c \
    "\copy (SELECT arabic, russian, transcription, part_of_speech FROM words) TO
'data/export.csv' WITH (FORMAT csv, HEADER true)"

# MariaDB
mariadb dict_ar_ru -e "
SELECT arabic, russian, transcription, part_of_speech
INTO OUTFILE '/tmp/export.csv'
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '\"'
LINES TERMINATED BY '\n'
FROM words;
"
```

Практика

> **ПРАКТИКА К ГЛАВЕ 17**
>
> Создайте CSV-файл с 20-30 словами (можно взять готовый из открытых словарей). Используйте скрипт `import_csv.sh` для импорта в вашу СУБД. Проверьте, что все записи добавились. Сделайте экспорт обратно в CSV и сравните с оригиналом (командой `diff`). Попробуйте импортировать тот же файл второй раз — должны возникнуть дубликаты. Добавьте защиту от дубликатов (UNIQUE constraint или `INSERT OR IGNORE` в SQLite).

ГЛАВА 18 · ОШИБКИ

Обработка ошибок и безопасность

Любойproduction-скрипт должен корректно обрабатывать ошибки. По умолчанию Bash продолжает выполнение после сбоя команды, что часто приводит к трудно диагностируемым багам: скрипт «успешно» завершается, но данные повреждены, файлы частично обновлены, база в несогласованном состоянии. В этой главе разберём практики, которые делают Bash-скрипты надёжными.

Коды возврата и $?

Каждая команда возвращает числовой код от 0 до 255: 0 означает успех, ненулевое — ошибку. Код последней команды сохраняется в переменной `$?`. Проверяйте его после критичных операций.

ПРОВЕРКА КОДА ВОЗВРАТА — BASH

```bash
sqlite3 "$SQLITE_DB" "INSERT INTO words VALUES (...);"
if [[ $? -ne 0 ]]; then
    log ERROR "Не удалось вставить запись"
    exit 1
fi

# Или коротко с помощью ||
sqlite3 "$SQLITE_DB" "INSERT ..." || {
    log ERROR "Ошибка вставки"
    exit 1
}
```

set -e, set -u, set -o pipefail

Три опции, которые должен включать каждый production-скрипт:

- **`set -e`** — немедленно выйти, если любая команда вернёт ненулевой код. Без этого скрипт продолжит работу даже после ошибки.
- **`set -u`** — ошибка при обращении к неопределённой переменной. Ловит опечатки в именах переменных.
- **`set -o pipefail`** — код возврата конвейера равен коду первой упавшей команды, а не последней. Без этого `false | true` вернёт 0 (успех).

Обычно их объединяют в одной строке в начале скрипта:

SET -EUO PIPEFAIL BASH

```bash
#!/bin/bash
set -euo pipefail
#等同于: set -e; set -u; set -o pipefail

# Этот скрипт упадёт, если:
# - любая команда вернёт ненулевой код
# - будет обращение к неопределённой переменной
# - любая команда в конвейере упадёт

source config.sh
dict_add "$1" "$2" "$3"
echo "Успешно добавлено"
```

Иногда `set -e` слишком агрессивна: вы хотите, чтобы команда могла упасть, и вы обработаете это. В таком случае используйте `|| true` или временно отключайте `-e`:

ИСКЛЮЧЕНИЯ ДЛЯ SET -E BASH

```bash
# Команда может упасть — это нормально
grep "что-то" file.txt || true      # grep возвращает 1, если нет совпадений
rm -f /tmp/tempfile 2>/dev/null || true

# Временно отключить
set +e
some_dangerous_command
result=$?
set -e

if (( result ≠ 0 )); then
    echo "Обработано"
fi
```

trap: очистка при выходе

Команда `trap` задаёт команду, которая выполнится при получении сигнала (EXIT, INT, TERM, ERR). Полезна для удаления временных файлов, закрытия соединений с БД, отката транзакций.

TRAP — ОЧИСТКА BASH

```bash
#!/bin/bash
set -euo pipefail

TMP_FILE=$(mktemp)
trap 'rm -f "$TMP_FILE"' EXIT
# При любом выходе (нормальном, по ошибке, по Ctrl+C)
# временный файл будет удалён

# Ловушка на ERR — для логирования ошибок
trap 'echo "Ошибка на строке $LINENO" >&2' ERR

# Используем временный файл
sqlite3 data/dict.db "SELECT * FROM words" > "$TMP_FILE"
wc -l "$TMP_FILE"
```

Специальные переменные в trap: `$LINENO` (номер строки), `$BASH_COMMAND` (команда, которая упала), `$?` (код возврата). Полный пример обработчика ошибок:

ПОЛНЫЙ ОБРАБОТЧИК ОШИБОК BASH

```bash
on_error() {
    local exit_code=$?
    local line=$1
    local cmd=$2
    echo "ОШИБКА: команда '$cmd' на строке $line вернула $exit_code" >&2
    exit $exit_code
}
trap 'on_error $LINENO "$BASH_COMMAND"' ERR

# Альтернатива: просто логирование
trap 'echo "Скрипт завершился с кодом $?" >> logs/dict.log' EXIT
```

Логирование

Каждая важная операция должна логироваться. Лог позволяет事后 понять, что произошло, и отладить проблемы. Хороший лог содержит: timestamp, уровень (INFO/WARN/ERROR), сообщение.

ФУНКЦИЯ ЛОГИРОВАНИЯ BASH

```bash
log() {
    local level="$1"; shift
    local msg="$*"
    local ts=$(date +'%Y-%m-%d %H:%M:%S')
    local script=$(basename "${BASH_SOURCE[1]}")
    local line="${BASH_LINENO[0]}"

    echo "[$ts] [$level] [$script:$line] $msg" >> "$LOG_DIR/dict.log"

    case "$level" in
        ERROR) echo -e "${CLR_RED}[$level] $msg${CLR_RESET}" >&2 ;;
        WARN)  echo -e "${CLR_YELLOW}[$level] $msg${CLR_RESET}" >&2 ;;
        INFO)  [[ "${VERBOSE:-0}" == "1" ]] && echo "[$level] $msg" ;;
    esac
}

# Использование
log INFO "Запуск скрипта"
log WARN "Файл пустой: $file"
log ERROR "Не удалось подключиться к БД"
VERBOSE=1 log INFO "Отладочное сообщение"
```

Валидация ввода

Никогда не доверяйте пользовательскому вводу. Проверяйте: что аргументы переданы, что файлы существуют, что строки соответствуют ожидаемому формату.

ВАЛИДАЦИЯ АРГУМЕНТОВ BASH

```bash
validate_args() {
    if [[ $# -lt 2 ]]; then
        echo "Использование: $0 <arabic> <russian> [transcription]" >&2
        exit 1
    fi

    local ar="$1" ru="$2"

    # Арабское слово: только арабские буквы
    [[ "$ar" =~ ^[\u0600-\u06FF]+$ ]] || {
        echo "Ошибка: '$ar' не похоже на арабское слово" >&2
        exit 1
    }

    # Русское слово: только кириллица и дефис
    [[ "$ru" =~ ^[а-яА-ЯёЁ-]+$ ]] || {
        echo "Ошибка: '$ru' не похоже на русское слово" >&2
        exit 1
    }

    # Проверка длины
    (( ${#ar} > 100 )) && {
        echo "Ошибка: слишком длинное арабское слово" >&2
        exit 1
    }
}

validate_args "$@"
```

Защита от SQL-инъекций

SQL-инъекция — это атака, когда злоумышленник вставляет SQL-код в данные. Пример: пользователь вводит «книга'); DROP TABLE words;--». Если скрипт просто подставляет это в SQL, таблица будет удалена. Защита: экранирование или параметризованные запросы.

ЗАЩИТА ОТ SQL-ИНЪЕКЦИЙ BASH

```bash
# Плохо: прямая подстановка
sqlite3 data/dict.db "SELECT * FROM words WHERE arabic = '$user_input';"
# Если $user_input = "'; DROP TABLE words; --"
# Получится: SELECT * FROM words WHERE arabic = ''; DROP TABLE words; --';

# Хорошо: экранирование одинарных кавычек
db_escape() {
    echo "${1//\'/\'\'}"   # ' → ''
}
local safe=$(db_escape "$user_input")
sqlite3 data/dict.db "SELECT * FROM words WHERE arabic = '$safe';"

# Ещё лучше: использовать .param в SQLite или $1 в PostgreSQL
# SQLite:
sqlite3 data/dict.db <<SQL
.param set :ar "$user_input"
SELECT * FROM words WHERE arabic = :ar;
SQL

# PostgreSQL:
psql -v ar="$user_input" -c "SELECT * FROM words WHERE arabic = :'ar';"
```

Для команды `dict_update` из главы 16 критично использовать белый список полей, иначе злоумышленник может передать `"id = 1; DROP TABLE"` как имя поля.

Безопасность файлов

ПРАВА НА ФАЙЛЫ BASH

```bash
# Файлы с паролями — только владелец
chmod 600 ~/.pgpass ~/.my.cnf

# Скрипты — исполняемые
chmod 700 scripts/*.sh

# БД — владелец читает/пишет, остальные — ничего
chmod 600 data/dict.db

# В скрипте проверка прав
[[ $(stat -c %a "$CONFIG_FILE") == "600" ]] || {
    echo "Небезопасные права на $CONFIG_FILE. Исправьте: chmod 600 $CONFIG_FILE"
>&2
    exit 1
}

# Безопасное создание временного файла
TMP=$(mktemp)              # в /tmp, случайное имя, права 600
trap 'rm -f "$TMP"' EXIT

# Не используйте: TMP=/tmp/myapp.$$  — уязвимо к race condition
```

ShellCheck: статический анализатор

ShellCheck — это линтер для Bash-скриптов, который находит типичные ошибки: незакавыченные переменные, использование `cat` там, где можно обойтись перенаправлением, устаревший синтаксис. Установите: `sudo apt install shellcheck`. Проверьте: `shellcheck myscript.sh`.

```
user@debian:~/dict-ar-ru$ sudo apt install shellcheck
user@debian:~/dict-ar-ru$ shellcheck scripts/dict.sh
In scripts/dict.sh line 15:
    echo $1
         ^-- SC2086: Double quote to prevent globbing
              and word splitting.

user@debian:~/dict-ar-ru$ shellcheck -x scripts/dict.sh   # с подключаемыми
файлами
```

Исправляйте все замечания ShellCheck — это значительно повысит качество скриптов. В CI/CD скрипты без прохода ShellCheck не должны попадать в production.

Практика

> **ПРАКТИКА К ГЛАВЕ 18**
>
> Добавьте `set -euo pipefail` и `trap` для очистки во все скрипты проекта. Реализуйте функцию `log` и используйте её везде вместо `echo` для отладочных сообщений. Запустите ShellCheck на всех скриптах и исправьте замечания. Попробуйте передать потенциально опасный ввод в `dict.sh add` (например, слово с одинарной кавычкой) и убедитесь, что экранирование работает.

ГЛАВА 19 · ПРОЕКТ

Полный проект словаря и бэкап

В этой главе соберём всё изученное в единый проект, который можно поддерживать и развивать. Мы добавим автоматические бэкапы, настройку cron, восстановление из резервной копии и финальные штрихи: README, LICENSE, git-репозиторий.

Финальная структура

```
dict-ar-ru/
├── README.md                  # Документация
├── LICENSE                    # Лицензия (MIT)
├── .gitignore                 # Что исключить из git
├── scripts/
│   ├── dict.sh                # Главный CLI
│   ├── config.sh              # Конфигурация
│   ├── import_csv.sh          # Импорт CSV
│   ├── export_csv.sh          # Экспорт CSV
│   ├── backup.sh              # Бэкап БД
│   ├── restore.sh             # Восстановление из бэкапа
│   └── lib/
│       ├── db.sh              # Абстракция БД
│       └── dict.sh            # Операции словаря
├── data/
│   ├── dict.db                # БД (или параметры подключения)
│   ├── schema_sqlite.sql
│   ├── schema_pg.sql
│   ├── schema_my.sql
│   └── words.csv              # Стартовые данные
├── backup/                    # Бэкапы (в .gitignore)
├── logs/                      # Логи (в .gitignore)
└── docs/
    └── arabic_unicode.md      # Заметки по арабскому в Unicode
```

Скрипт бэкапа

SCRIPTS/BACKUP.SH BASH

```bash
#!/bin/bash
set -euo pipefail
source "$(dirname "$0")/config.sh"

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="$PROJECT_ROOT/backup"
mkdir -p "$BACKUP_DIR"

log() { echo "[$(date +'%H:%M:%S')] $*"; }

case "$DB_ENGINE" in
    sqlite)
        FILE="$BACKUP_DIR/dict_$TIMESTAMP.db"
        # Способ 1: простое копирование файла
        cp "$SQLITE_DB" "$FILE"
        # Способ 2: через .dump (SQL-текст, восстанавливается в любой SQLite)
        sqlite3 "$SQLITE_DB" .dump > "$BACKUP_DIR/dict_$TIMESTAMP.sql"
        log "Бэкап SQLite: $FILE и .sql"
        ;;
    postgresql)
        FILE="$BACKUP_DIR/dict_$TIMESTAMP.sql.gz"
        pg_dump -U "$PG_USER" -d "$PG_DB" | gzip > "$FILE"
        log "Бэкап PostgreSQL: $FILE"
        ;;
    mariadb)
        FILE="$BACKUP_DIR/dict_$TIMESTAMP.sql.gz"
        mysqldump --defaults-file="$HOME/.my.cnf" "$MY_DB" | gzip > "$FILE"
        log "Бэкап MariaDB: $FILE"
        ;;
esac

# Удаление старых бэкапов (старше 30 дней)
find "$BACKUP_DIR" -name "dict_*" -mtime +30 -delete
log "Старые бэкапы удалены"
```

Скрипт восстановления

SCRIPTS/RESTORE.SH BASH

```bash
#!/bin/bash
set -euo pipefail
source "$(dirname "$0")/config.sh"

BACKUP_FILE="$1"

[[ ! -f "$BACKUP_FILE" ]] && {
    echo "Файл не найден: $BACKUP_FILE" >&2
    exit 1
}

echo "Внимание: текущие данные будут заменены. Продолжить? [y/N]"
read -r ans
[[ "$ans" =~ ^[Yy]$ ]] || exit 0

case "$DB_ENGINE" in
    sqlite)
        if [[ "$BACKUP_FILE" == *.sql ]]; then
            rm -f "$SQLITE_DB"
            sqlite3 "$SQLITE_DB" < "$BACKUP_FILE"
        else
            cp "$BACKUP_FILE" "$SQLITE_DB"
        fi
        ;;
    postgresql)
        gunzip -c "$BACKUP_FILE" | psql -U "$PG_USER" -d "$PG_DB"
        ;;
    mariadb)
        gunzip -c "$BACKUP_FILE" | mariadb "$MY_DB"
        ;;
esac

echo "Восстановлено из $BACKUP_FILE"
```

Cron: автоматические бэкапы

Cron — системный планировщик задач. Запускает команды по расписанию. Чтобы настроить ежедневный бэкап в 3 часа ночи, отредактируйте crontab:

```
user@debian:~$ crontab -e
# Открыт редактор (обычно nano). Добавьте строку:

# Ежедневный бэкап в 3:00
0 3 * * * /home/user/dict-ar-ru/scripts/backup.sh >> /home/user/dict-ar-ru/logs/cron.log 2>&1

# Еженедельный экспорт в CSV по воскресеньям в 4:00
0 4 * * 0 /home/user/dict-ar-ru/scripts/export_csv.sh >> /home/user/dict-ar-ru/logs/cron.log 2>&1

# Сохранить и выйти (в nano: Ctrl+O, Enter, Ctrl+X)

user@debian:~$ crontab -l   # проверить
```

Формат cron: `минута час день_месяца месяц день_недели команда`. Звёздочка — любое значение. `*/5` — каждые 5 минут. `0 3 * * *` — каждый день в 3:00. `0 4 * * 0` — каждое воскресенье в 4:00.

Git: версионирование

GIT ИНИЦИАЛИЗАЦИЯ BASH

```bash
cd ~/dict-ar-ru
git init
git config user.name "Your Name"
git config user.email "you@example.com"

# .gitignore
cat > .gitignore <<EOF
backup/
logs/
*.db
*.log
*.swp
.DS_Store
EOF

git add .
git commit -m "Initial commit: dict-ar-ru v1.0"

# Опционально: загрузить на GitHub
git remote add origin git@github.com:user/dict-ar-ru.git
git push -u origin main
```

README

README.MD MARKDOWN

```
# Арабо-русский словарь

Bash-проект для управления арабо-русским словарём в Debian GNU/Linux.
Поддерживает SQLite, PostgreSQL и MariaDB.

## Установка

1. Склонируйте репозиторий: `git clone ...`
2. Установите SQLite: `sudo apt install sqlite3`
3. Примените схему: `sqlite3 data/dict.db < data/schema_sqlite.sql`
4. Импортируйте стартовые данные: `./scripts/import_csv.sh`
5. Проверьте: `./scripts/dict.sh list`

## Использование

    ./scripts/dict.sh add كتاب книга kitab существительное
    ./scripts/dict.sh search книга
    ./scripts/dict.sh list
    ./scripts/dict.sh count

## Переменные окружения

- `DB_ENGINE` — sqlite (по умолчанию) | postgresql | mariadb
- `VERBOSE` — 1 для отладочного вывода

## Бэкап

    ./scripts/backup.sh
    ./scripts/restore.sh backup/dict_20260123_030000.db

## Лицензия

MIT
```

Тестирование

Простой скрипт для самопроверки проекта:

SCRIPTS/TEST.SH BASH

```bash
#!/bin/bash
set -euo pipefail
source "$(dirname "$0")/config.sh"

pass=0; fail=0

test_assert() {
    local desc="$1"
    local expected="$2"
    local actual="$3"
    if [[ "$expected" == "$actual" ]]; then
        ((pass++))
        echo "  ✓ $desc"
    else
        ((fail++))
        echo "  ✗ $desc (ожидалось '$expected', получено '$actual')"
    fi
}

echo "=== Тесты словаря ==="

# Тест: добавление и поиск
./scripts/dict.sh add тесттест testtest testtext существительное 2>/dev/null
result=$(./scripts/dict.sh search testtext | grep -c тесттест)
test_assert "Добавление и поиск" "1" "$result"

# Тест: count
before=$(./scripts/dict.sh count | grep -oE '[0-9]+')
./scripts/dict.sh add тест2 test2 test2 существительное 2>/dev/null
after=$(./scripts/dict.sh count | grep -oE '[0-9]+')
test_assert "Увеличение count" "$((before + 1))" "$after"

echo
echo "=== Итог: $pass passed, $fail failed ==="
exit $((fail > 0 ? 1 : 0))
```

Практика

> **ПРАКТИКА К ГЛАВЕ 19**
>
> Доведите проект до финального состояния: создайте `backup.sh`, `restore.sh`, `test.sh`, README, .gitignore. Инициализируйте git-репозиторий и сделайте первый коммит. Настройте cron для ежедневного бэкапа. Запустите `test.sh` — все тесты должны проходить.

ГЛАВА 20 · ИТОГИ

Итоги и дальнейшее изучение

Поздравляем! Вы прошли путь от первой команды в терминале Debian до полноценного Bash-проекта арабо-русского словаря. Давайте подведём итоги и наметим направления для дальнейшего развития.

Что вы освоили

За двадцать глав вы изучили основы Bash: переменные и их типы, перенаправления потоков, конвейеры, here-documents, условия и циклы, функции и рекурсию. Вы научились работать с текстом через `grep`, `sed`, `awk` и регулярные выражения. Вы настроили Debian для корректного ввода и отображения арабского текста: установили шрифты, раскладку клавиатуры, локаль. Вы сравнили три популярные СУБД — SQLite, PostgreSQL и MariaDB — и реализовали слой абстракции, позволяющий переключаться между ними одной переменной окружения. Вы освоили обработку ошибок, логирование, защиту от SQL-инъекций и автоматические бэкапы.

Главный навык, который вы приобрели — это не знание конкретных команд, а умение комбинировать маленькие инструменты в сложные pipeline. Unix-философия: «пиши программы, которые делают одну вещь, но делают её хорошо, и программы, которые работают вместе» — лежит в основе всего, что мы изучили. `grep` ищет, `sed` редактирует, `awk` обрабатывает поля, `sqlite3` хранит данные. Соединённые через конвейеры, они решают задачи любой сложности.

Книги для углубления

«Advanced Bash-Scripting Guide» (Mendel Cooper) — исчерпывающее руководство по Bash от основ до продвинутых тем. Доступно бесплатно на tldp.org. Обязательное чтение для тех, кто хочет писать сложные скрипты.

«Bash Cookbook» (Carl Albing, JP Vossen) — практические рецепты для типичных задач: обработка текста, системное администрирование, сетевое программирование. Каждая глава — конкретная задача и её решение.

« sed & awk» (Dale Dougherty) — классическая книга по двум мощнейшим текстовым утилитам. Хотя книга старая, материал не устарел, поскольку `sed` и `awk` практически не изменились за 30 лет.

«SQL за 10 минут» (Ben Forta) — короткое практическое введение в SQL. Подойдёт, если хотите углубить знания по базам данных.

Онлайн-ресурсы

- **tldp.org** — The Linux Documentation Project, руководства HOWTO и руководства по Bash.
- **shellcheck.com** — онлайн-версия линтера ShellCheck, проверяет скрипты в браузере.
- **man bash** — официальное руководство. Длинное, но исчерпывающее. Особое внимание разделам `SHELL BUILTINS`, `PARAMETERS`, `EXPANSION`.
- **www.gnu.org/software/bash/manual** — официальное руководство GNU Bash, с гиперссылками и поиском.
- **sqlite.org/docs.html** — документация SQLite, особенно разделы по FTS (полнотекстовый поиск) и ICU.
- **postgresql.org/docs** — официальная документация PostgreSQL, одна из лучших в индустрии.
- **habr.com/ru/hub/linux** — статьи на русском по Linux и Bash.

Идеи для развития проекта

Созданный нами словарь — это фундамент. Вот несколько направлений, как его можно развить.

Веб-интерфейс. Превратите словарь в веб-приложение. Простой вариант: CGI-скрипт на Bash, который через `printf "Content-Type: text/html\n\n"` выводит HTML. Серьёзный вариант: Flask (Python) или Express (Node.js) как фронтенд, оставляя нашу SQLite-БД как хранилище. Это откроет словарь для доступа из браузера и с телефона.

Полнотекстовый поиск. SQLite FTS5 и PostgreSQL tsvector дают мощный поиск с ранжированием по релевантности, морфологией, стоп-словами. Для арабского нужна дополнительная конфигурация (Arabic stemmer), но результат того стоит.

Мобильное приложение. React Native или Flutter позволяют переиспользовать SQLite-БД на мобильных устройствах. Словарь в кармане — отличный продукт для изучающих арабский.

Экспорт в Anki. Anki — популярная программа для интервального повторения. Экспортируйте словарь в формат Anki-карточек (TSV с разделителями), и пользователи смогут учить слова с эффективностью интервального повторения.

API. Опубликуйте REST API для доступа к словарю: `GET /api/words?q=книга`. Это позволит другим разработчикам создавать поверх вашего словаря свои приложения.

Нормализация арабского. Реализуйте функции нормализации: удаление огласовок, унификация алифов (ا → إ أ آ), та марбута (ة → ه), проверка хамзы. Это сделает поиск более толерантным к вариантам написания.

Импорт из внешних словарей. Существуют открытые арабо-английские словари (Wiktionary, dict.cc). Напишите парсеры, которые загружают их в вашу БД с автоматическим переводом на русский через API.

Сообщество

Программирование в одиночку быстро выгорает. Найдите единомышленников: Telegram-чаты по Bash и Linux, форумы Stack Overflow (теги `bash`, `shell`), Хабр. Участвуйте в open-source: найдите проект на GitHub по теме словаря или Bash-инструментов, поправьте мелкие баги, отправьте pull request. Это лучший способ прокачать навыки и завести профессиональные связи.

Debian-сообщество — одно из самых дружелюбных. Подпишитесь на debian-user@lists.debian.org, заходите на #debian в IRC или Matrix. Если ваш проект окажется полезным, рассмотрите возможность создания собственного Debian-пакета — это новый уровень навыков.

Финальные советы

Регулярность важнее интенсивности: 30 минут Bash каждый день за месяц дадут больше, чем 15 часов в воскресенье. Делайте pet-проекты — решайте собственные задачи, автоматизируйте рутину. Не бойтесь ошибок: каждый баг, который вы исправили, делает вас сильнее. Читайте чужой код: лучшие Bash-скрипты лежат в `/usr/bin` и `/usr/sbin` вашего Debian — открывайте и изучайте.

И последнее. Программирование — это не про знание команд, а про умение решать задачи. Команды забываются, навык поиска ответа остаётся. Учитесь гуглить эффективно, формулировать вопросы, читать документацию. Эти мета-навыки важнее любого конкретного языка или утилиты.

Удачи в дальнейшем изучении Bash, Debian и баз данных. Ваш арабо-русский словарь — это первый шаг в мир системного программирования, который длится всю жизнь. Пусть он будет полезным и принесёт радость и вам, и другим изучающим арабский язык.

ПРИЛОЖЕНИЕ · СПРАВКА

Шпаргалка по командам

Краткий справочник по основным командам и конструкциям, использованным в книге. Сохраните в закладки или распечатайте для быстрого доступа.

Файлы и каталоги

КОМАНДА	ОПИСАНИЕ
`pwd`	Текущий каталог
`cd path`	Сменить каталог
`ls -la`	Список файлов с правами
`mkdir -p a/b/c`	Создать вложенные каталоги
`touch file`	Создать пустой файл
`cp -r src dst`	Копировать рекурсивно
`mv old new`	Переместить/переименовать
`rm -rf dir`	Удалить рекурсивно без подтверждения
`chmod 755 file`	Установить права rwxr-xr-x
`chown user:group file`	Сменить владельца
`find . -name "*.txt"`	Поиск файлов по шаблону
`ln -s target link`	Символическая ссылка

Просмотр содержимого

КОМАНДА	ОПИСАНИЕ
`cat file`	Вывести весь файл
`less file`	Постраничный просмотр
`head -n 10 file`	Первые 10 строк
`tail -n 10 file`	Последние 10 строк
`tail -f log`	Следить за файлом
`wc -l file`	Количество строк
`file file`	Определить тип файла
`stat file`	Подробная информация

Текстовые утилиты

КОМАНДА	ОПИСАНИЕ
`grep -irn "pattern" .`	Рекурсивный поиск с номерами строк
`sed -i 's/old/new/g' file`	Замена в файле на месте
`awk -F, '{print $1}' file.csv`	Первая колонка CSV
`cut -d: -f1 /etc/passwd`	Вырезать поле
`sort -u file`	Сортировка с уникализацией
`uniq -c`	Подсчёт повторов (после sort)
`tr 'a-z' 'A-Z'`	Перевод в верхний регистр
`paste a b`	Объединить файлы построчно
`tee file`	Дублировать stdin в файл и stdout
`xargs -I{} cmd {}`	Подставить строки как аргументы

Bash-конструкции

КОНСТРУКЦИЯ	ОПИСАНИЕ
`var="value"`	Присваивание (без пробелов!)
`${var}` или `$var`	Чтение переменной
`$(cmd)`	Командная подстановка
`$((expr))`	Арифметика
`${var:-default}`	Значение по умолчанию
`${#var}`	Длина строки
`${var//old/new}`	Замена подстроки
`[[ -f file ]]`	Проверка существования файла
`[[ "$s" =~ ^[a-z]+$ ]]`	Regex совпадение
`if ...; then ...; fi`	Условие
`for x in list; do ...; done`	Цикл по списку
`while read line; do ...; done < file`	Построчное чтение
`case x in a) ...;; esac`	Выбор по шаблону
`func() { ...; }`	Функция
`local var`	Локальная переменная
`set -euo pipefail`	Строгий режим
`trap 'cmd' EXIT`	Команда при выходе

Специальные переменные

ПЕРЕМЕННАЯ	ЗНАЧЕНИЕ
`$0`	Имя скрипта
`$1..$9`	Аргументы 1-9
`${10}`	Аргумент 10+
`$#`	Количество аргументов
`$@`	Все аргументы (массив)
`$*`	Все аргументы (строка)
`$?`	Код возврата последней команды
`$$`	PID текущего процесса
`$!`	PID последнего фонового процесса
`$LINENO`	Номер текущей строки
`$BASH_SOURCE`	Имя текущего файла скрипта
`$HOME, $USER, $PWD`	Системные переменные

Команды БД

ДЕЙСТВИЕ	SQLITE	POSTGRESQL	MARIADB
Установка	`apt install sqlite3`	`apt install postgresql`	`apt install mariadb-server`
Клиент	`sqlite3 db.db`	`psql -d db`	`mariadb db`
Выполнить SQL	`sqlite3 db "SQL"`	`psql -c "SQL"`	`mariadb -e "SQL"`
Из файла	`sqlite3 db < f.sql`	`psql -f f.sql`	`mariadb db < f.sql`
Дамп	`sqlite3 db .dump`	`pg_dump db`	`mysqldump db`
Список таблиц	`.tables`	`\dt`	`SHOW TABLES;`
Схема	`.schema`	`\d table`	`DESCRIBE table;`
Выход	`.exit`	`\q`	`exit`

Финальный скрипт dict.sh — полный текст

Для справки — компактная версия главного скрипта со всеми подкомандами. Полная версия с библиотеками — в репозитории проекта.

SCRIPTS/DICT.SH (КОМПАКТНАЯ ВЕРСИЯ) BASH

```bash
#!/bin/bash
set -euo pipefail

# Конфигурация
export LC_ALL=ru_RU.UTF-8
export DB_ENGINE="${DB_ENGINE:-sqlite}"
export SQLITE_DB="$HOME/dict-ar-ru/data/dict.db"
export PG_DB="dict_ar_ru" PG_USER="$USER"
export MY_DB="dict_ar_ru"

db_query() {
    case "$DB_ENGINE" in
        sqlite)     sqlite3 "$SQLITE_DB" "$1" ;;
        postgresql) psql -t -U "$PG_USER" -d "$PG_DB" -c "$1" ;;
        mariadb)    mariadb -N "$MY_DB" -e "$1" ;;
    esac
}

db_pretty() {
    case "$DB_ENGINE" in
        sqlite)     sqlite3 -header -column "$SQLITE_DB" "$1" ;;
        postgresql) psql -U "$PG_USER" -d "$PG_DB" -c "$1" ;;
        mariadb)    mariadb -t "$MY_DB" -e "$1" ;;
    esac
}

db_escape() { echo "${1//\'/\'\'}"; }

usage() {
    cat <<EOF
Использование: dict.sh <команда> [аргументы]
Команды: add search list count delete update help
СУБД: DB_ENGINE=sqlite|postgresql|mariadb (по умолчанию sqlite)
EOF
}

[[ $# -eq 0 ]] && { usage; exit 0; }

case "$1" in
    add)
        ar=$(db_escape "$2"); ru=$(db_escape "$3")
        tr=$(db_escape "${4:-}"); pos=$(db_escape "${5:-}")
        db_query "INSERT INTO words (arabic, russian, transcription,
part_of_speech)
            VALUES ('$ar', '$ru', '$tr', '$pos');"
        echo "Добавлено: $2 → $3"
        ;;
    search)
```

Этот скрипт — отправная точка. Расширяйте, дополняйте, делайте свой словарь полезным для себя и других. Удачи!